



Digital IC Design

Sequential Logic and Finite State Machine with examples in Verilog

Taihai Chen, 陈泰海
taihai.chen@zju.edu.cn

2024.12.31

Today's arrangement



1. Sequential logic with examples in Verilog
2. Finite State Machine with examples in Verilog
3. Live demos in Vivado
4. Q & A

In between these...

Interaction is strongly recommended.

If you have questions? Please ask anytime.

Note:

1. Teaching materials of my part in this course are mostly referenced from and inspired by Prof. Yubo Li (李宇波教授). So, thank you Prof Li!
2. Some of RTL code in this course are referenced from and/or inspired by www.vlsiverify.com and 《从算法到电路: 数字芯片算法的电路实现》 (*From algorithm to circuits: algorithms and implementation for digital chips* in English).

Description of a Flip-Flop



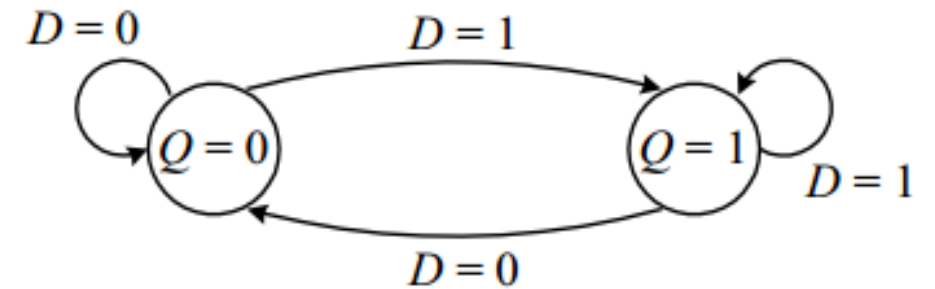
- (a) Characteristic Table
- (b) Characteristic Equation
- (c) State Diagram

D	Q	Q_{next}
0	$\times$	0
1	$\times$	1

(a)

$$Q_{\text{next}} = D$$

(b)

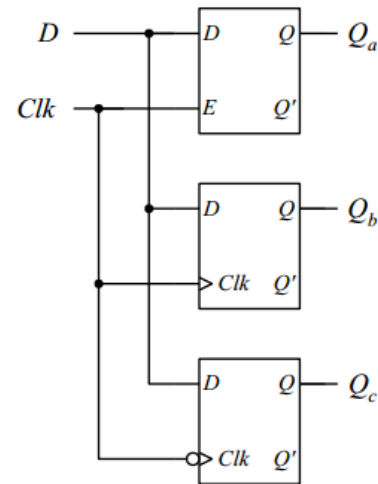


(c)

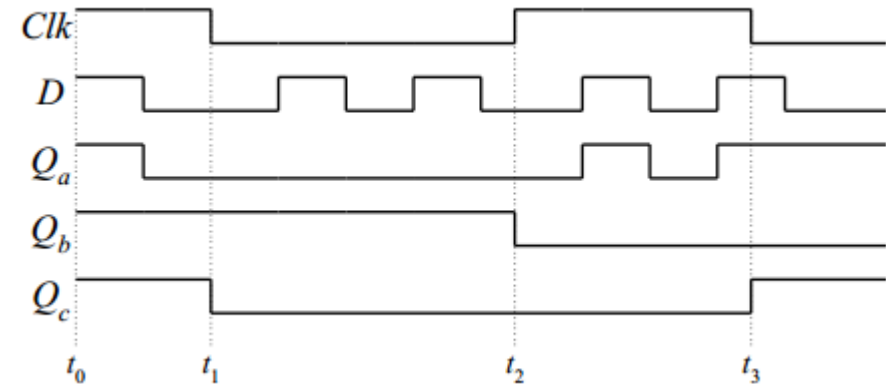
- Behavioral Verilog code for a D flip-flop

```
module D_flipflop (  
    input D,  
    input Clock,  
    output reg Q  
);  
  
    // execute on rising clock edge  
    always @(posedge Clock) begin  
        Q <= D;          // assign value from D to Q at every rising clock edge  
    end  
  
endmodule
```

- Comparison of
 - Gated latch
 - positive edge-triggered flip-flop
 - negative edge-triggered flip-flop



(a) circuit

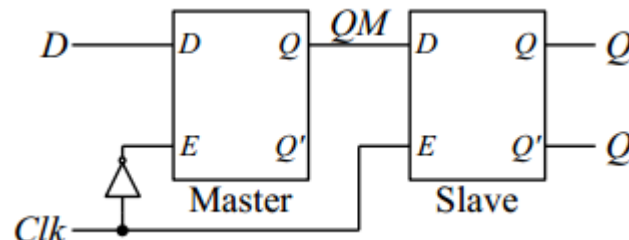


(b) sample trace

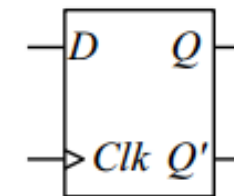
D Flip-Flop



- Master-slave positive edge-triggered D flip-flop



(a)

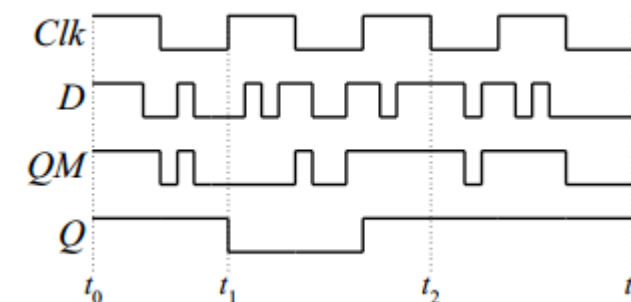


(b)

- (a) circuit
- (b) logic symbol
- (c) operation table
- (d) sample trace

Clk	D	Q	Q_{next}	Q_{next}'
0	x	0	0	1
0	x	1	1	0
1	x	0	0	1
1	x	1	1	0
↑	0	x	0	1
↑	1	x	1	0

(c)

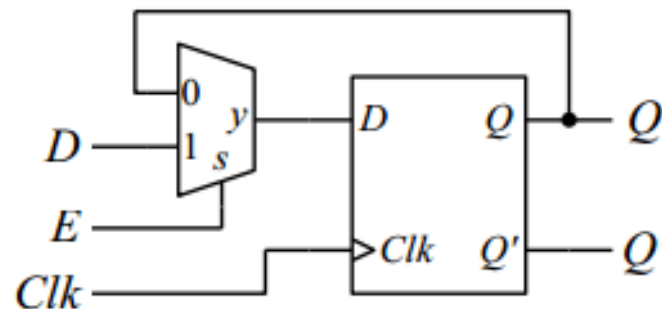


(d)

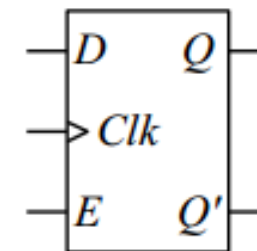
D Flip-Flop with Enable



- D Flip-flop with enable
 - (a) circuit
 - (b) logic symbol
 - (c) operation table



(a)

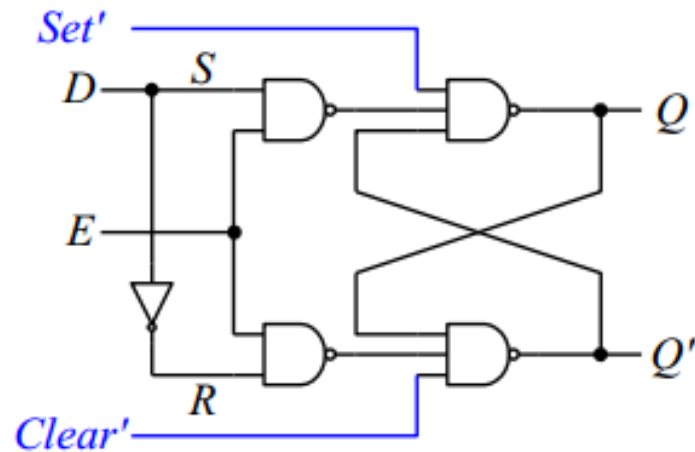


(c)

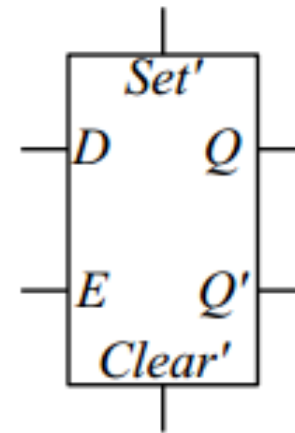
Clk	E	D	Q	Q_{next}	Q_{next}'
0	×	×	0	0	1
0	×	×	1	1	0
1	×	×	0	0	1
1	×	×	1	1	0
↑	0	×	0	0	1
↑	0	×	1	1	0
↑	1	0	×	0	1
↑	1	1	×	1	0

(c)

- Storage elements with asynchronous inputs
 - (a) D latch with active-low set and clear
 - (b) logic symbol for (a)



(a)



(b)

D Flip-Flop with Enable



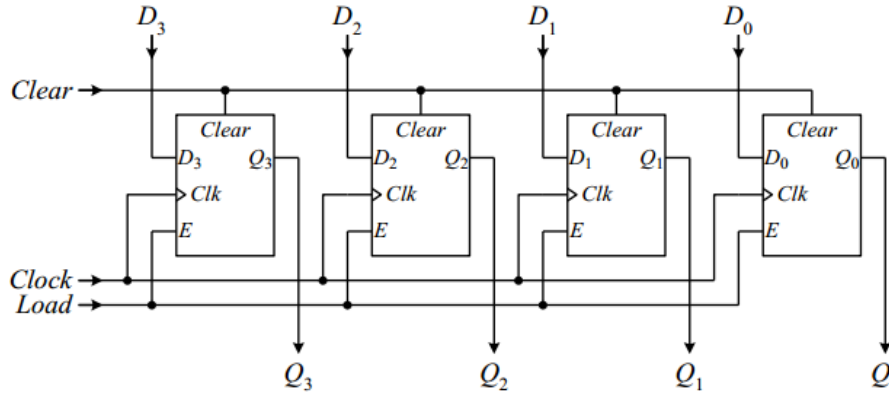
- Behavioral Verilog code for a D flip-flop with synchronous enable and asynchronous clear

```
module D_flipflop (
    input D,                      // signals
    input Clock,
    input Enable,
    input Clear,
    output reg Q
);

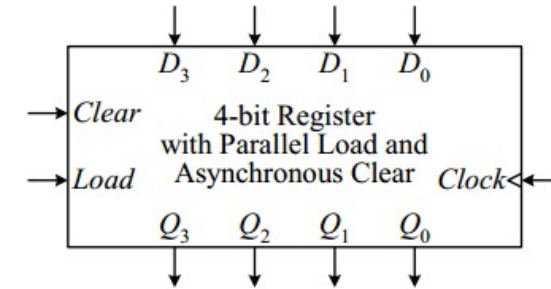
    // execute on rising clock edge or Clear
    always @(posedge Clock or posedge Clear) begin
        if (Clear) begin
            Q <= 1'b0;           // assign 0 to Q on clear
        end else if (Enable) begin
            Q <= D;               // assign value from D to Q only if Enable is asserted
        end
    end
end

endmodule
```

- A 4-bit register with synchronous load and asynchronous clear
 - (a) circuit
 - (b) logic symbol
 - (c) operation table



(a)



(b)

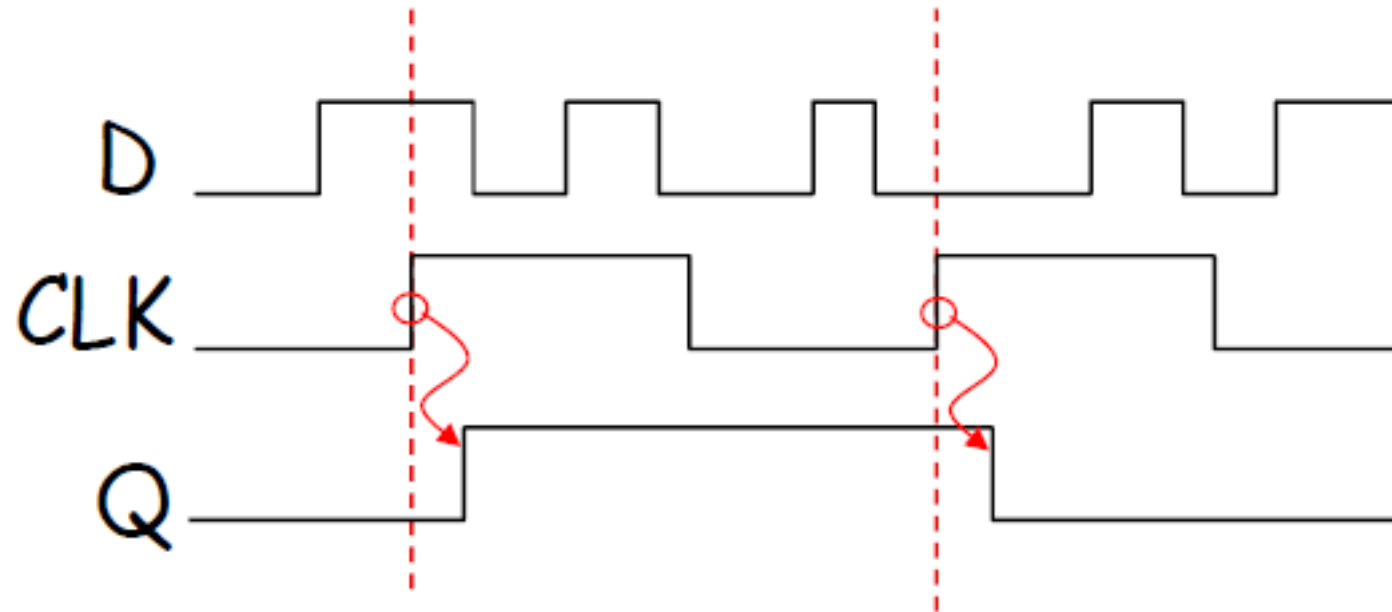
<i>Clear</i>	<i>Load</i>	Operation
1	×	Reset register to zero asynchronously
0	0	No change
0	1	Load in <i>D</i> value at rising clock edge

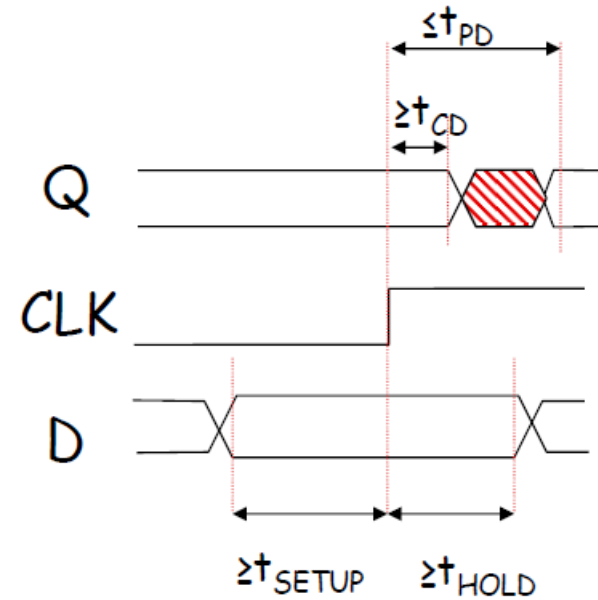
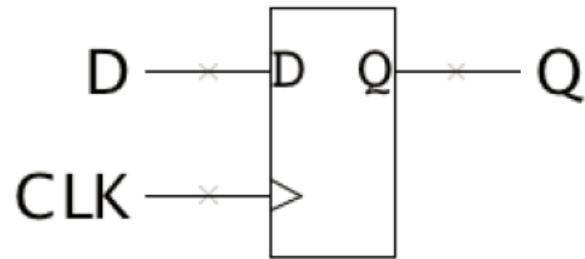
(c)

- Behavioral Verilog code for a 4-bit register with active-high *Load* and *Clear* signals

```
module reg (  
    input Clock, Clear, Load,  
    input [n-1:0] D,  
    output reg [n-1:0] Q  
);  
  
    parameter n = 4; // allow n to be changed  
  
    always @ (posedge Clock or posedge Clear) begin  
        if (Clear == 1) begin  
            Q <= 0;  
        end else if (Clock == 1) begin  
            if (Load == 1) begin  
                Q <= D;  
            end  
        end  
    end  
  
endmodule
```

DFF Timing





t_{PD} : maximum propagation delay, $CLK \rightarrow Q$

t_{CD} : minimum contamination delay, $CLK \rightarrow Q$

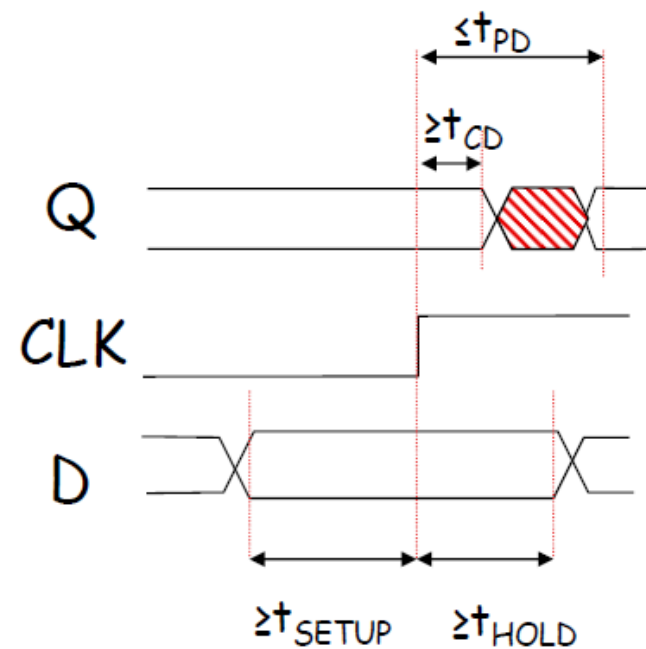
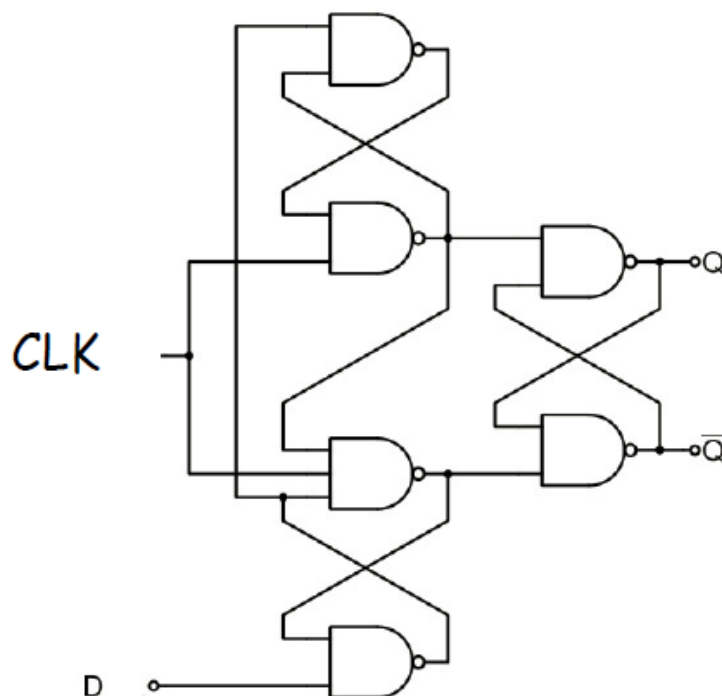
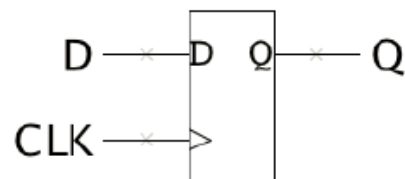
t_{SETUP} : setup time

How long D must be stable before the rising edge of CLK

t_{HOLD} : hold time

How long D must be stable after the rising edge of CLK

DFF Internals – 74LS74

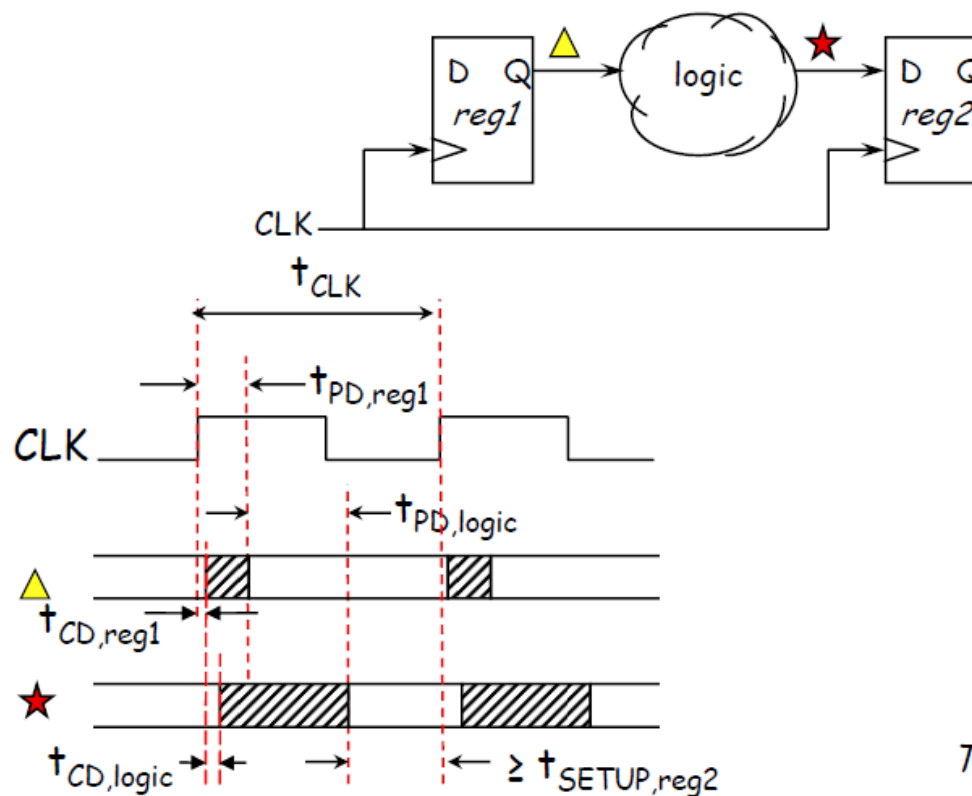


$$t_{\text{SETUP}} = 20\text{ns} \quad t_{\text{PD-HL}} = 40\text{ns}$$

$$t_{\text{HOLD}} = 5\text{ns} \quad t_{\text{PD-LH}} = 25\text{ns}$$

DFF can not remember. Why is it still so popular in digital design?

DFF Timing



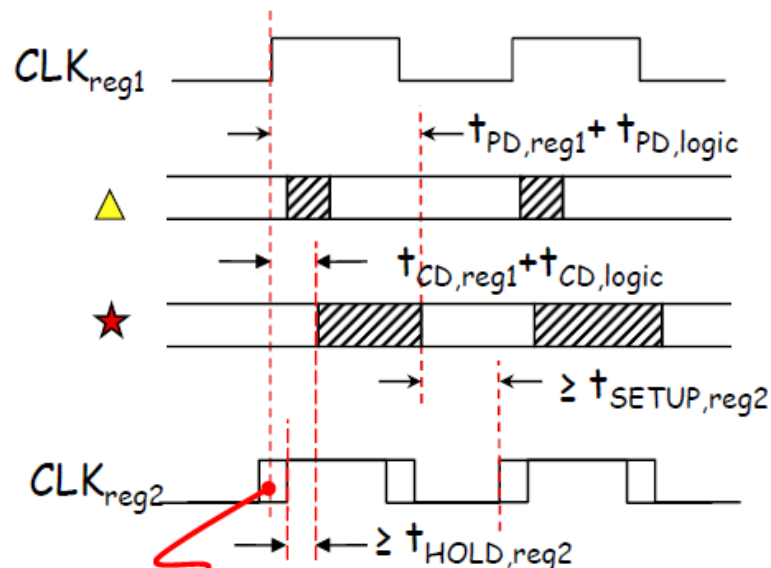
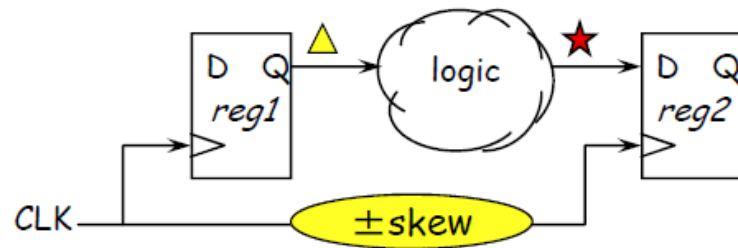
The good news: you can choose t_{CLK} so that this constraint is satisfied!

$$t_{PD,reg1} + t_{PD,logic} + t_{SETUP,reg2} \leq t_{CLK}$$

The bad news: you have to change your design if this constraint isn't met.

$$t_{CD,reg1} + t_{CD,logic} \geq t_{HOLD,reg2}$$

DFF Timing with Skew



CLK_{reg2} rising edge might fall anywhere in this region.

Which skew is tougher to deal with (pos or neg)?

In the real world the clock signal arrives at different registers at different times. The difference in arrival times (pos or neg) is called the **clock skew** t_{skew} .

$$t_{skew} = t_{Rn,clk2} - t_{Rn,clk1}$$

We can update our two timing constraints to reflect the worst-case skew

Setup time: $t_{Rn,clk} = t_{Rn+1,clk}$

$$t_{Rn,clk1} + t_{PD,reg1} + t_{PD,logic} + t_{SETUP,reg2} \leq t_{Rn+1,clk2}$$

$$t_{PD,reg1} + t_{PD,logic} + t_{SETUP,reg2} \leq t_{CLK} + t_{skew}$$

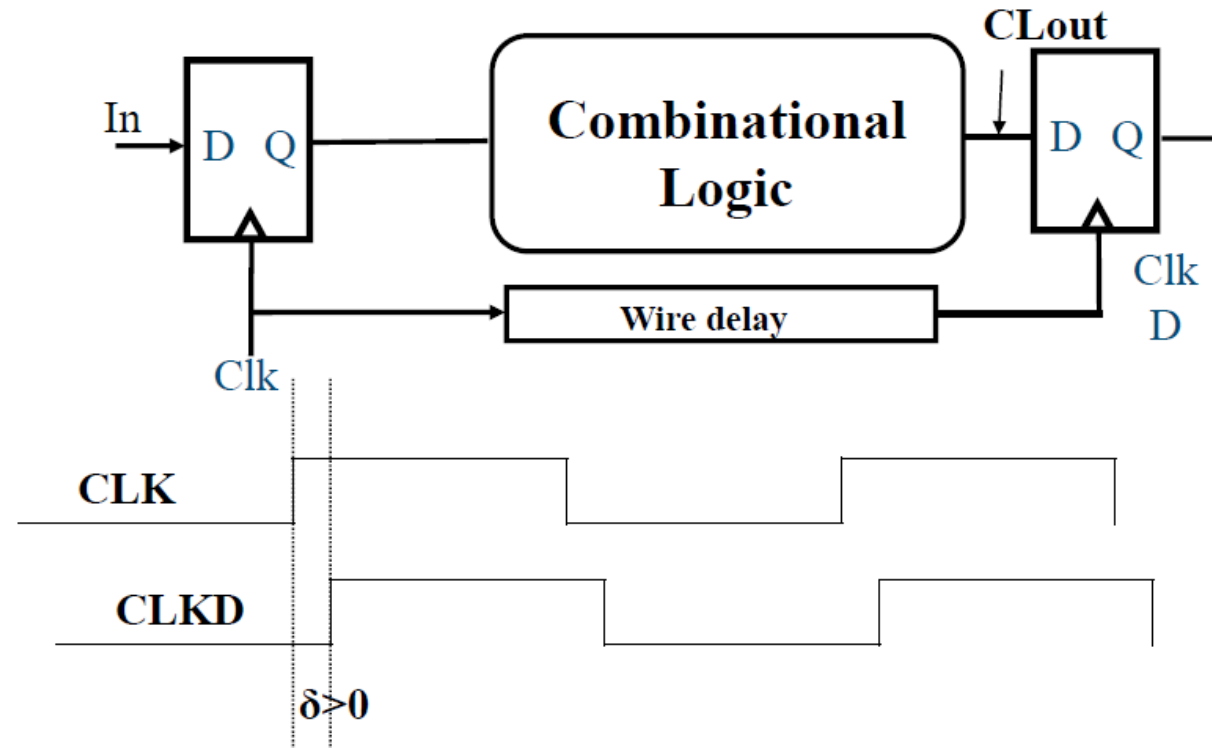
Hold time:

$$t_{Rn,clk1} + t_{CD,reg1} + t_{CD,logic} \geq t_{Rn,clk2} + t_{HOLD,reg2}$$

$$t_{CD,reg1} + t_{CD,logic} \geq t_{HOLD,reg2} + t_{skew}$$

Thus clock skew increases the minimum cycle time of our design and makes it harder to meet register hold times.

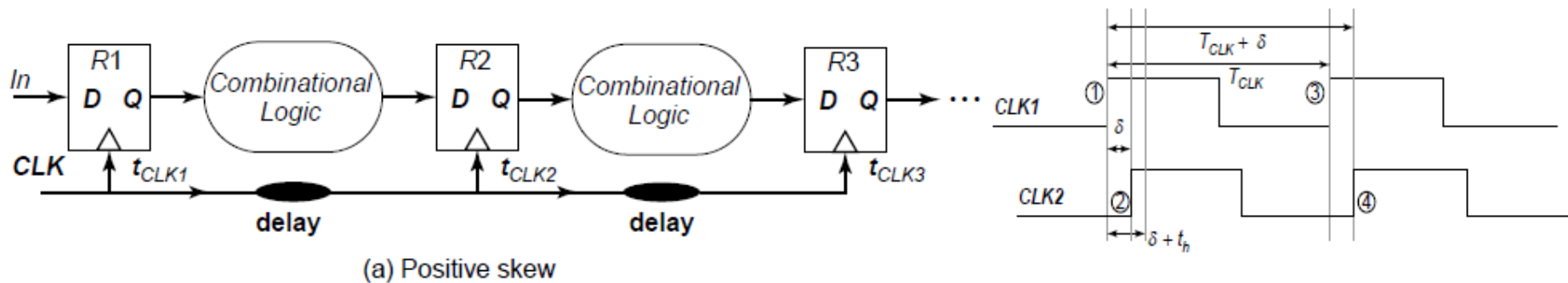
Clocks are not perfect: Clock Skew



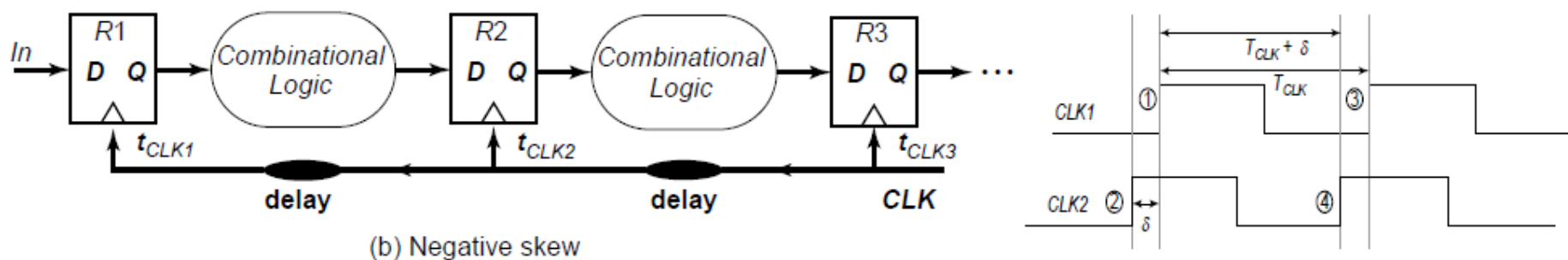
$$T > T_{cq} + T_{logic} + T_{su} - \delta$$

$$T_{cq,cd} + T_{logic,cd} > T_{hold} + \delta$$

Positive and Negative Skew

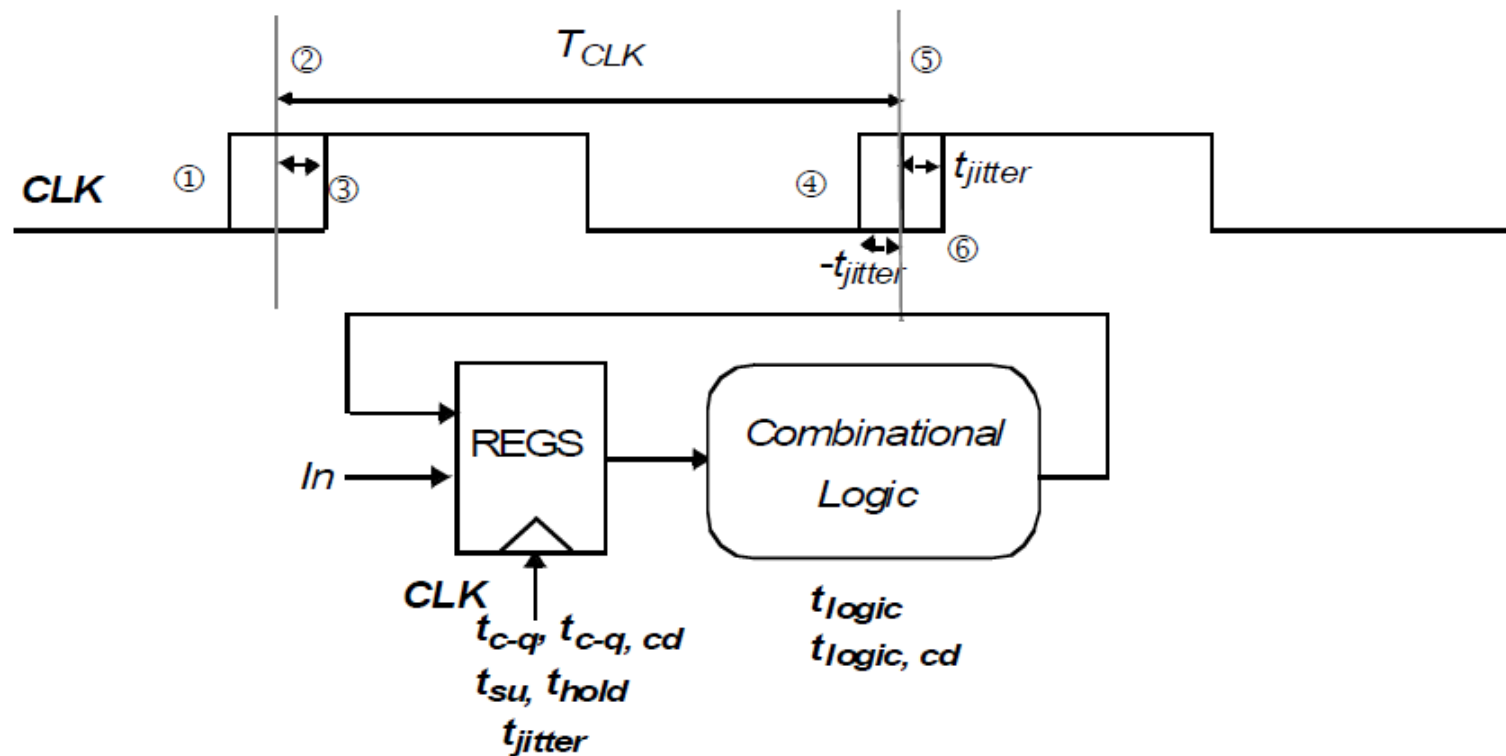


Launching edge arrives before the receiving edge



Receiving edge arrives before the launching edge

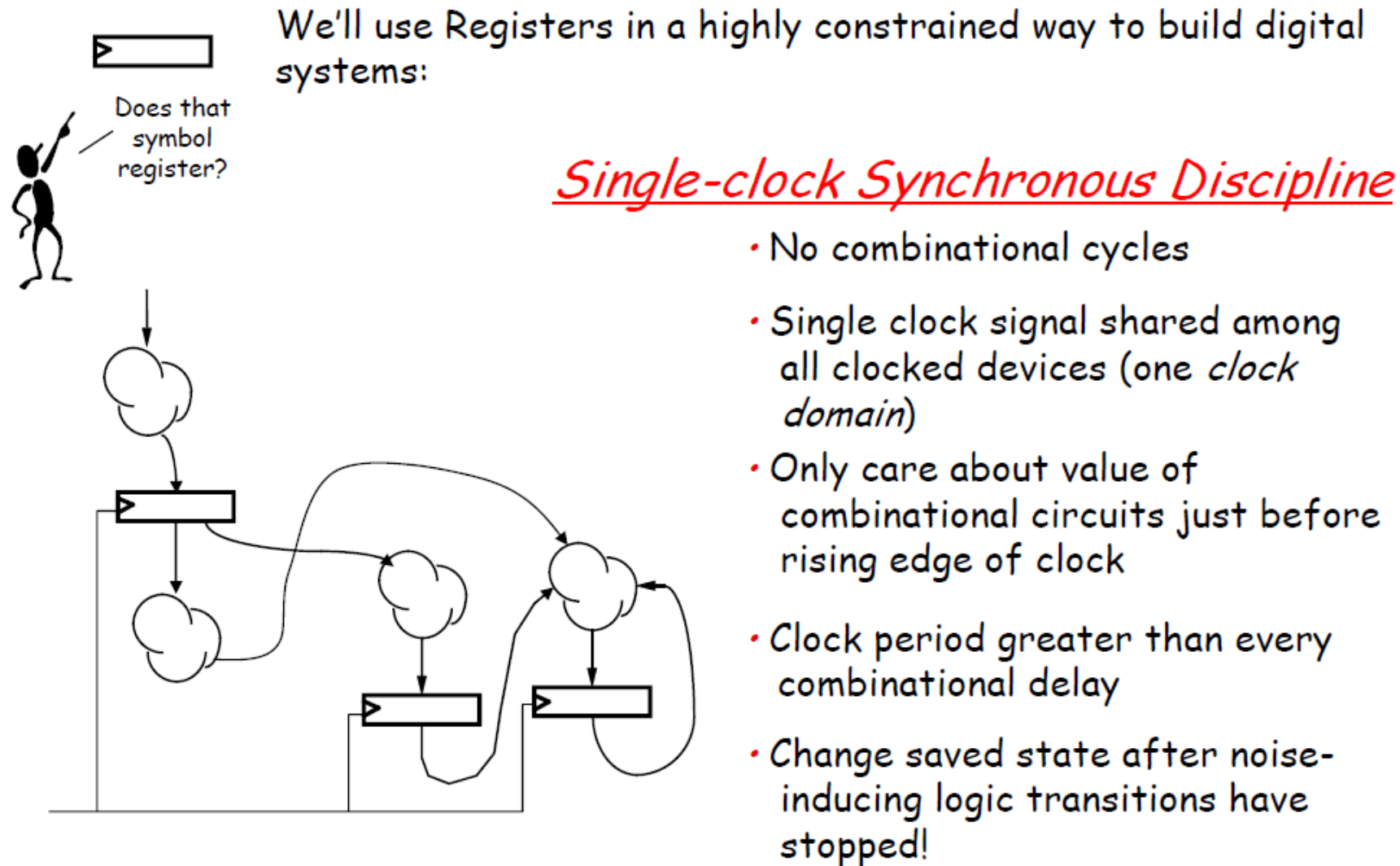
Clocks are not perfect: Clock Jitter



$$T_{CLK} - 2t_{jitter} > t_{c-q} + t_{logic} + t_{su}$$

or

$$T > t_{c-q} + t_{logic} + t_{su} + 2t_{jitter}$$



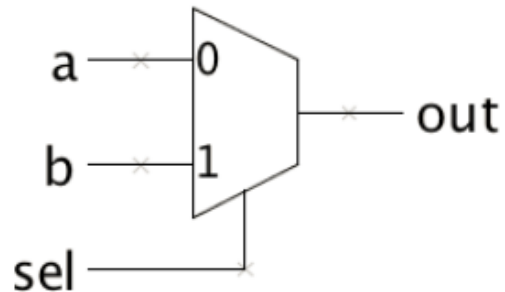
The Sequential always block



Edge-triggered circuits are described using a sequential `always` block

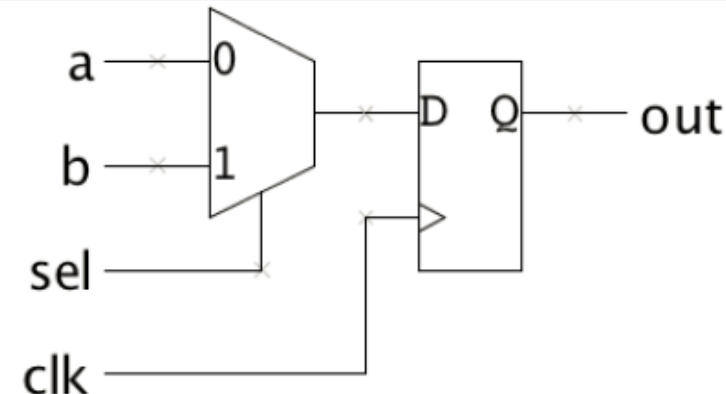
Combinational

```
module comb(input a, b, sel,
             output reg out);
  always @(*) begin
    if (sel) out = b;
    else out = a;
  end
endmodule
```



Sequential

```
module seq(input a, b, sel, clk,
           output reg out);
  always @(posedge clk) begin
    if (sel) out <= b;
    else out <= a;
  end
endmodule
```



Importance of Sensitive List



- The use of `posedge` and `negedge` makes an `always` block sequential (edge-triggered)
- Unlike a combinational `always` block, the sensitivity list **does** determine behavior for synthesis!

D-Register with **synchronous** clear

```
module dff_sync_clear(  
    input d, clearb, clock,  
    output reg q  
);  
    always @(posedge clock)  
    begin  
        if (!clearb) q <= 1'b0;  
        else q <= d;  
    end  
endmodule
```

`always` block entered only at each positive clock edge

D-Register with **asynchronous** clear

```
module dff_sync_clear(  
    input d, clearb, clock,  
    output reg q  
);  
    always @(negedge clearb or posedge clock)  
    begin  
        if (!clearb) q <= 1'b0;  
        else q <= d;  
    end  
endmodule
```

`always` block entered immediately when (active-low) `clearb` is asserted

Note: The following is incorrect syntax: `always @(clear or negedge clock)`

If one signal in the sensitivity list uses `posedge`/`negedge`, then all signals must.

- Assign any signal or variable from only one `always` block. Be wary of race conditions: `always` blocks with same trigger execute concurrently...

Blocking vs Nonblocking Assignment



- Verilog supports two types of assignments within `always` blocks, with subtly different behaviors.
- *Blocking assignment (=)*: evaluation and assignment are immediate

```
always @(*) begin
  x = a | b;      // 1. evaluate a|b, assign result to x
  y = a ^ b ^ c;  // 2. evaluate a^b^c, assign result to y
  z = b & ~c;     // 3. evaluate b&(~c), assign result to z
end
```

Nonblocking assignment (<=): all assignments deferred to end of simulation time step after all right-hand sides have been evaluated (*even those in other active `always` blocks*)

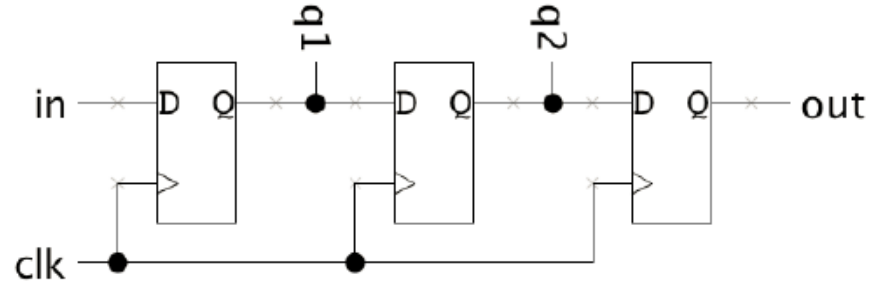
```
always @(*) begin
  x <= a | b;      // 1. evaluate a|b, but defer assignment to x
  y <= a ^ b ^ c;  // 2. evaluate a^b^c, but defer assignment to y
  z <= b & ~c;     // 3. evaluate b&(~c), but defer assignment to z
  // 4. end of time step: assign new values to x, y and z
end
```

Sometimes, as above, both produce the same result. **Sometimes, not!**

Assignment Style for Sequential Logic



*What we want:
Register Based
Digital Delay Line*



Will nonblocking and blocking assignments both produce the desired result? ("old" means value before clock edge, "new" means the value after most recent assignment)

```
module nonblocking(  
    input in, clk,  
    output reg out  
);  
    reg q1, q2;  
    always @(posedge clk) begin  
        q1 <= in;  
        q2 <= q1; // uses old q1  
        out <= q2; // uses old q2  
    end  
endmodule
```

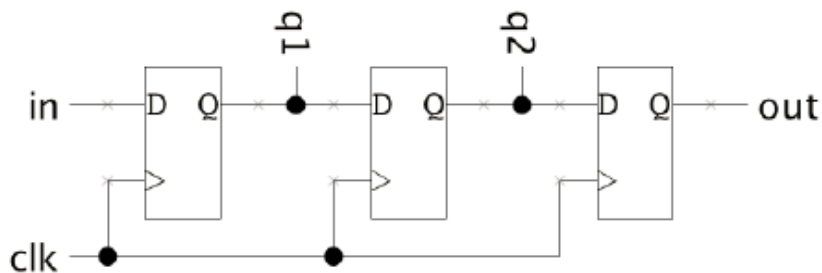
```
module blocking(  
    input in, clk,  
    output reg out  
);  
    reg q1, q2;  
    always @(posedge clk) begin  
        q1 = in;  
        q2 = q1; // uses new q1  
        out = q2; // uses new q2  
    end  
endmodule
```


Using Nonblocking for Sequential Logic



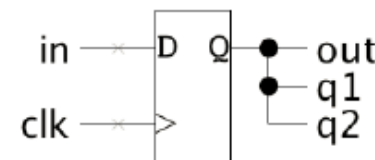
```
always @(posedge clk) begin
    q1 <= in;
    q2 <= q1;    // uses old q1
    out <= q2;   // uses old q2
end
```

"At each rising clock edge, $q1$, $q2$, and out **simultaneously receive the old values** of in , $q1$, and $q2$."



```
always @(posedge clk) begin
    q1 = in;
    q2 = q1;    // uses new q1
    out = q2;   // uses new q2
end
```

"At each rising clock edge, $q1 = in$.
After that, $q2 = q1$.
After that, $out = q2$.
Therefore $out = in$."



- Blocking assignments ***do not*** reflect the intrinsic behavior of multi-stage sequential logic
- **Guideline: use *nonblocking* assignments for sequential *always* blocks**

- Sequential always block: `always @(posedge clock)` **use <=**
- Combinatorial always block: `always @ *` **use =**
- Results of operators (LHS) inside always block (sequential and combinatorial) must be declared as “reg”
- Equivalent Verilog

```
reg z  
always @ *  
z = x && y
```

← same as →
example of
combinatorial
always block

```
assign z = x && y  
// z not a “reg”
```

- case statements must be used within an always block; include default case

Sequential Always Block Style



```
// There are two styles for creating this sample divider. The
// first uses sequential always block for state assignment and
// a combinational always block for next-state. This style tends
// to result in fewer errors.
//
// An alternate approach is to use a single always block. An example
// of a divide by 5 counter will illustrate the differences
```

```
////////////////////////////////////
// Sequential always block with a
// combinational always block

reg [3:0] count1, next_count1;

always @(posedge clk)
    count1 <= next_count1;

always @* begin
    if (reset) next_count1 = 0;
    else next_count1 =
        (count1 == 4) ? 0 : count1 + 1;
end

assign enable1 = (count1 == 4);
////////////////////////////////////
```

```
////////////////////////////////////
// Single always block
//

reg [3:0] count2;

always @(posedge clk) begin
    if (reset) count2 <= 0;
    else count2 <=
        (count2 == 4) ? 0 : count2 + 1;
end

assign enable2 = (count2 == 4);
////////////////////////////////////
```

Sequential and Combinational in the Same always Block



```
module nbex1
(output reg q,
 input clk, rst_n,
 input a, b);

reg y;
always @(a or b)
y = a ^ b;

always @(posedge clk or
        negedge rst_n)
if (!rst_n) q <= 1'b0;
else q <= y;

endmodule
```

Combinatorial logic

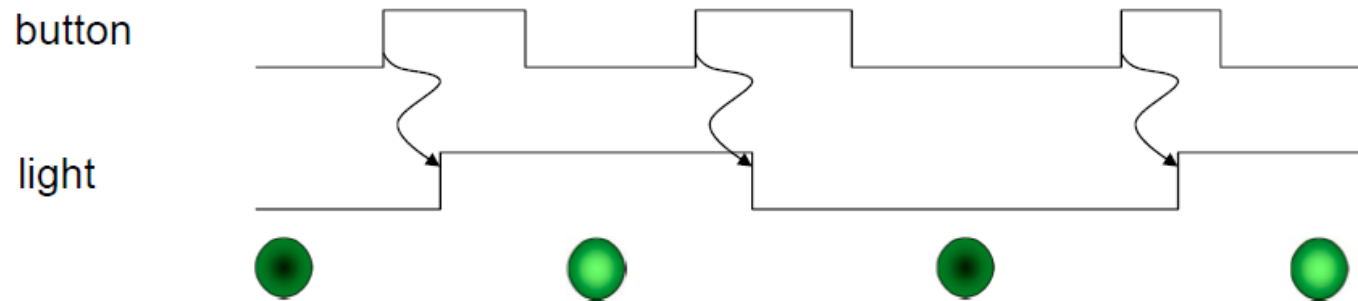
```
module nbex2
(output q,
 input clk, rst_n,
 input a, b);

reg q;
always @(posedge clk or
        negedge rst_n)
if (!rst_n) q <= 1'b0;
else q <= a ^ b;

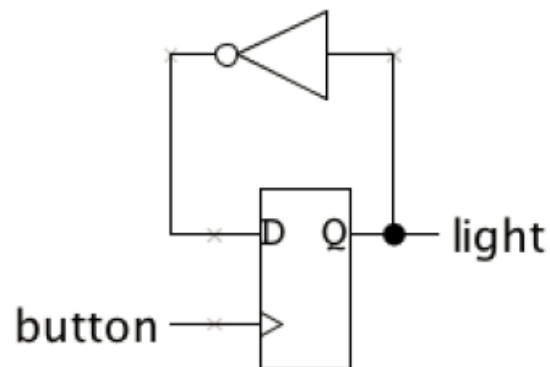
endmodule
```

Combinatorial logic

Implement an ON/OFF button



```
module onoff(input button, output reg light);  
    always @(posedge button) light <= ~light;  
endmodule
```



Synchronous ON/OFF Button



When designing a system that accepts many inputs it would be hard to have input changes serve as the system clock (which input would we use?). So we'll use a single clock of some fixed frequency and have the inputs control what state changes happen on rising clock edges.

For most of our lab designs we'll use a 27MHz system clock (37ns clock period).

```
module onoff_sync(input clk, button,  
                  output reg light);  
    always @ (posedge clk) begin  
        if (button) light <= ~light;  
    end  
endmodule
```

Usually one can't rely on registers powering-on to a particular initial state*. So most designs have a RESET signal that when asserted initializes all the state to known, mutually consistent initial values.

```
module onoff_sync(input clk, reset, button,
                  output reg light);
  always @ (posedge clk) begin
    if (reset) light <= 0;
    else if (button) light <= ~light;
  end
endmodule
```

* Actually, our FPGAs will reset all registers to 0 when the device is programmed. But it's nice to be able to press a reset button to return to a known state rather than starting from scratch by reprogramming the device.

Clocks are fast, we are slow



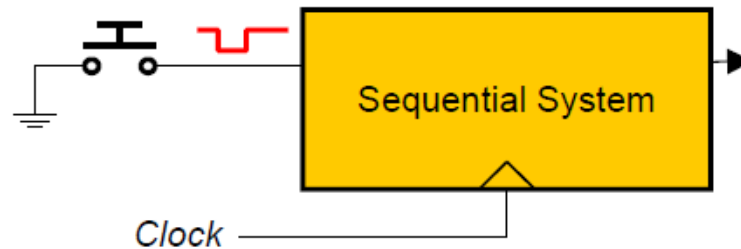
The circuit on the last slide toggles the light on every rising clock edge for which button is 1. But clocks are fast (27MHz!) and our fingers are slow, so how do we press the button for just one clock edge? Answer: we can't, but we can add some state that remembers what button was last clock cycle and then detect the clock cycles when button changes from 0 to 1.

```
module onoff_sync(input clk, reset, button,
                  output reg light);
    reg old_button; // state of button last clk
    always @ (posedge clk) begin
        if (reset)
            begin light <= 0; old_button <= 0; end
        else if (old_button==0 && button==1)
            // button changed from 0 to 1
            light <= ~light;
            old_button <= button;
        end
    endmodule
```


Asynchronous Input in Sequential Systems

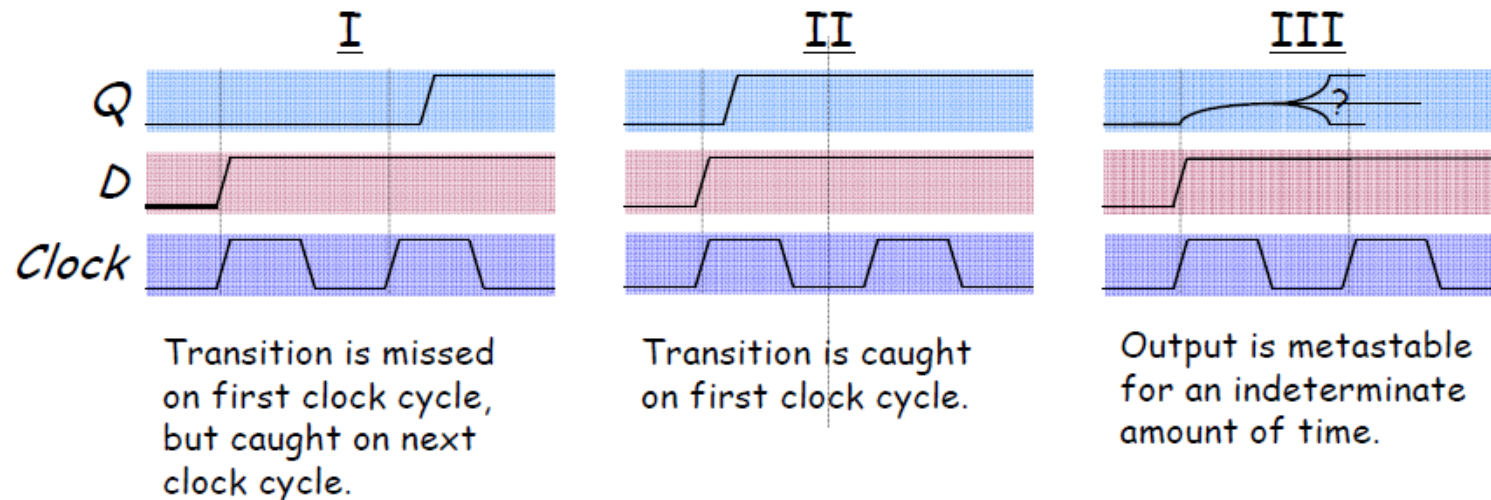


What about external signals?



*Can't guarantee
setup and hold
times will be met!*

When an asynchronous signal causes a setup/hold violation...



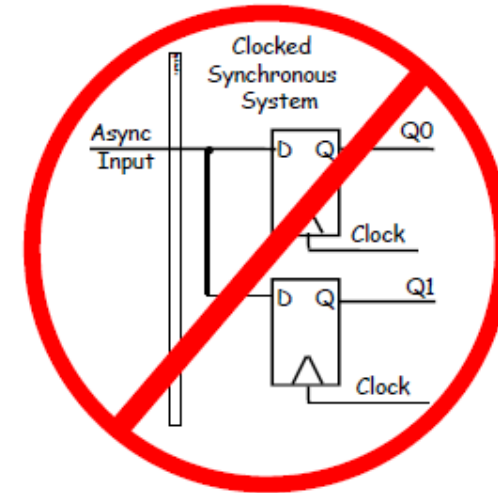
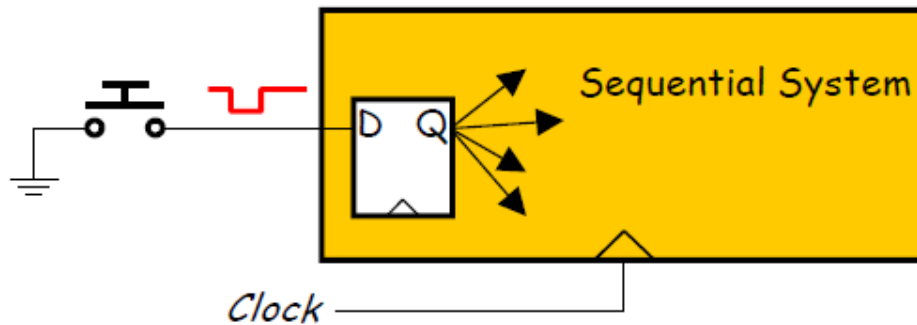
Q: Which cases are problematic?

Asynchronous Input in Sequential Systems



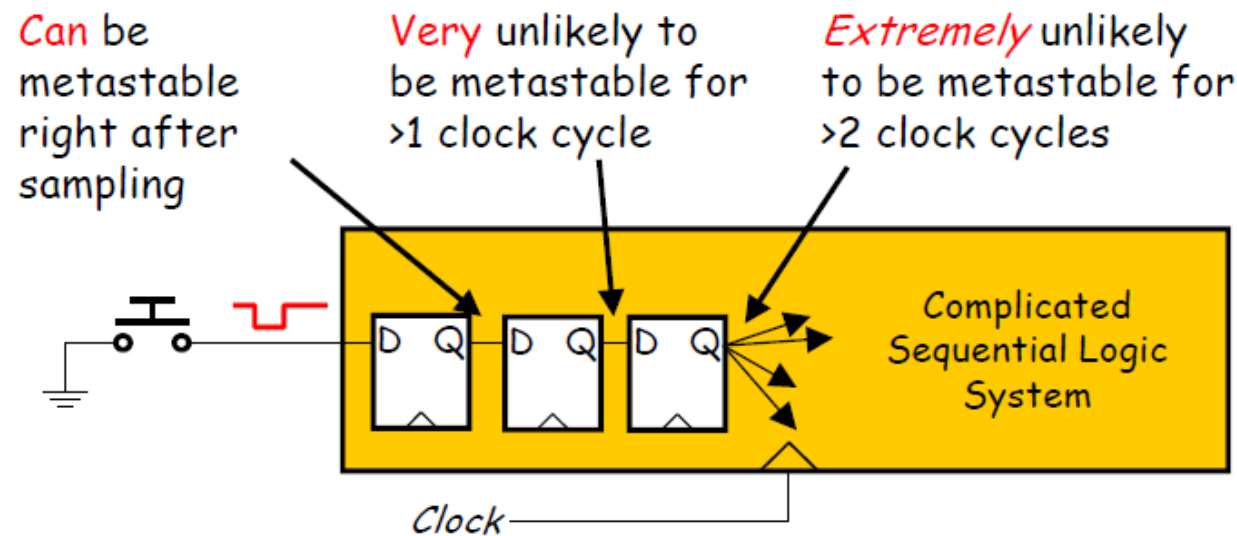
All of them can be, if more than one happens simultaneously within the same circuit.

Guideline: ensure that external signals directly feed exactly one flip-flop



This prevents the possibility of I and II occurring in different places in the circuit, but what about metastability?

- Preventing metastability turns out to be an impossible problem
- High gain of digital devices makes it likely that metastable conditions will resolve themselves quickly
- Solution to metastability: allow time for signals to stabilize



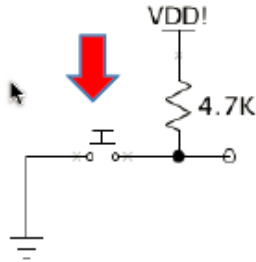
How many registers are necessary?

- Depends on many design parameters (clock speed, device speeds, ...)
- In 6.111, a pair of synchronization registers is sufficient

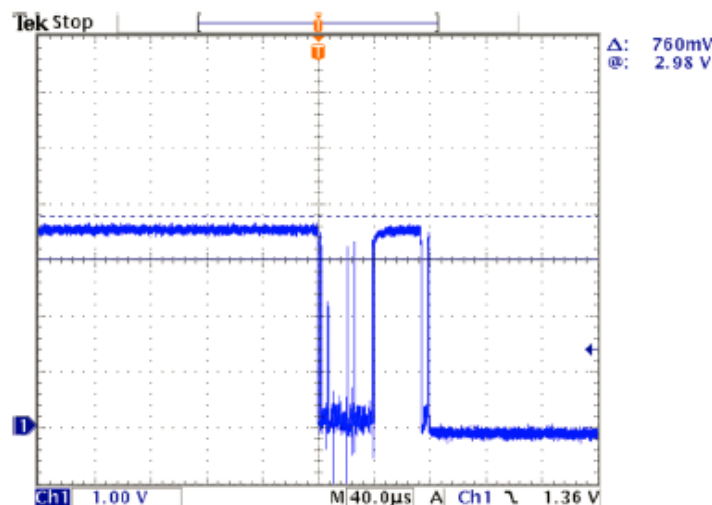
One Last Little Problem



Mechanical buttons exhibit contact "bounce" when they change position, leading to multiple output transitions before finally stabilizing in the new position:



We need a debouncing circuit!



```
module debounceIndex (reset, clk, noisy, clean);  
    input reset, clk, noisy;  
    output clean;
```

```
    parameter NDELAY = 650000;  
    parameter NBITS = 20;
```

```
    reg [NBITS-1:0] count;  
    reg xnew, clean;
```

```
    always @(posedge clk)  
        if (reset) begin
```

```
            xnew <= noisy; clean <= noisy; count <= 0;
```

```
        end
```

```
        else if (noisy != xnew) begin
```

```
            xnew <= noisy; count <= 0;
```

```
        end
```

```
        else if (count == NDELAY)
```

```
            clean <= xnew;
```

```
        else
```

```
            count <= count+1;
```

```
    endmodule
```

On/Off Button, Final Answer



```
module onoff_sync(input clk, reset, button_in,
                  output reg light);

    // synchronizer
    reg button, btemp;
    always @(posedge clk)
        {button, btemp} <= {btemp, button_in};

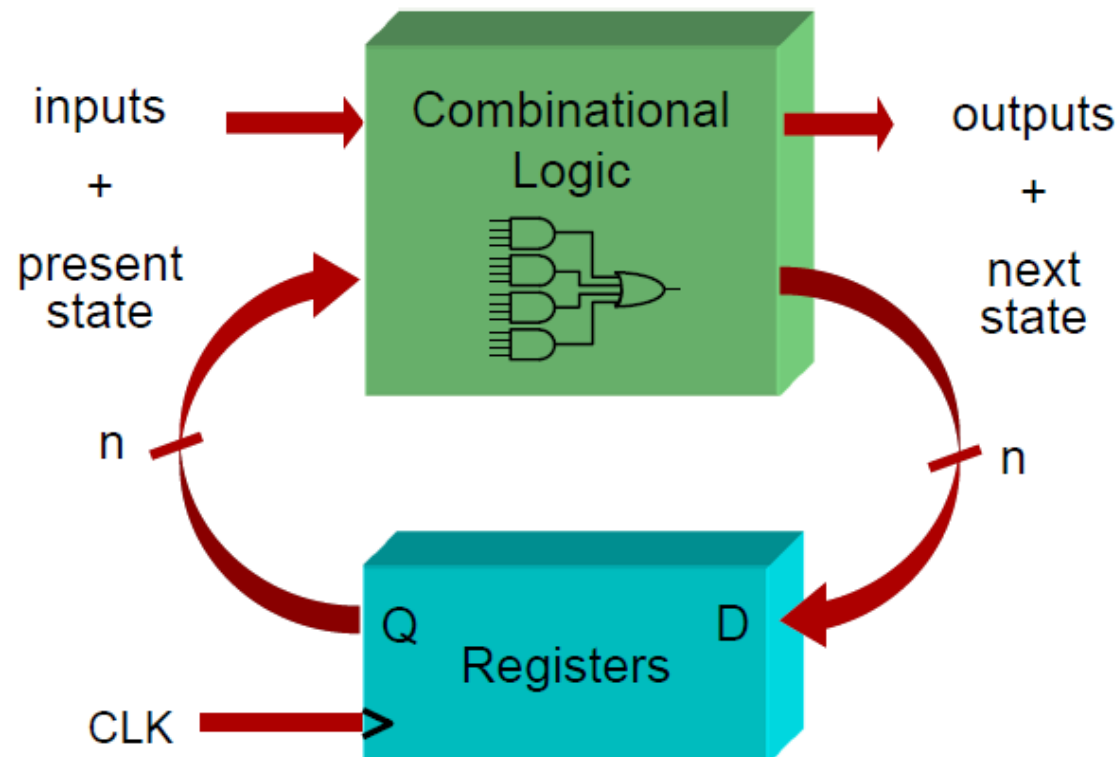
    // debounce push button
    wire bpressed;
    debounce db1(.clock(clk), .reset(reset),
                .boucey(button), .steady(bpressed));

    reg old_bpressed; // state last clk cycle
    always @ (posedge clk) begin
        if (reset)
            begin light <= 0; old_bpressed <= 0; end
        else if (old_bpressed==0 && bpressed==1)
            // button changed from 0 to 1
            light <= ~light;
        old_bpressed <= bpressed;
    end
endmodule
```



Finite State Machine

- Finite State Machines (FSMs) are a useful abstraction for *sequential circuits* with centralized “*states*” of operation
- At each clock edge, combinational logic computes *outputs* and *next state* as a function of *inputs* and *present state*



- The Finite-State Machine (FSM) is the controller circuit for all microprocessors
- It has a finite number of states
- It is deterministic, i.e. from the current state, it will know exactly what is the next state to go to
- It has both sequential and combinational circuits

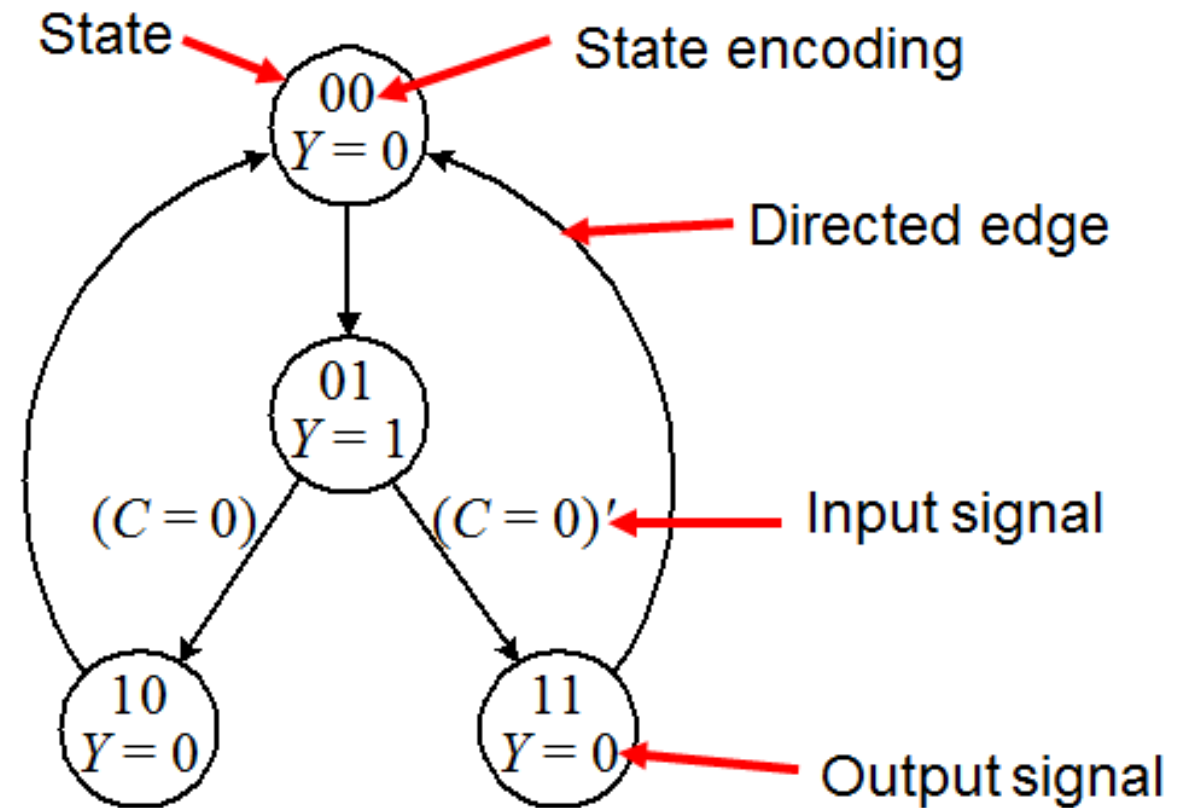


- Used to describe precisely the operation of a FSM
- A deterministic graph with nodes and directed edges connecting the nodes
- Edges are either conditional or unconditional

State Diagram



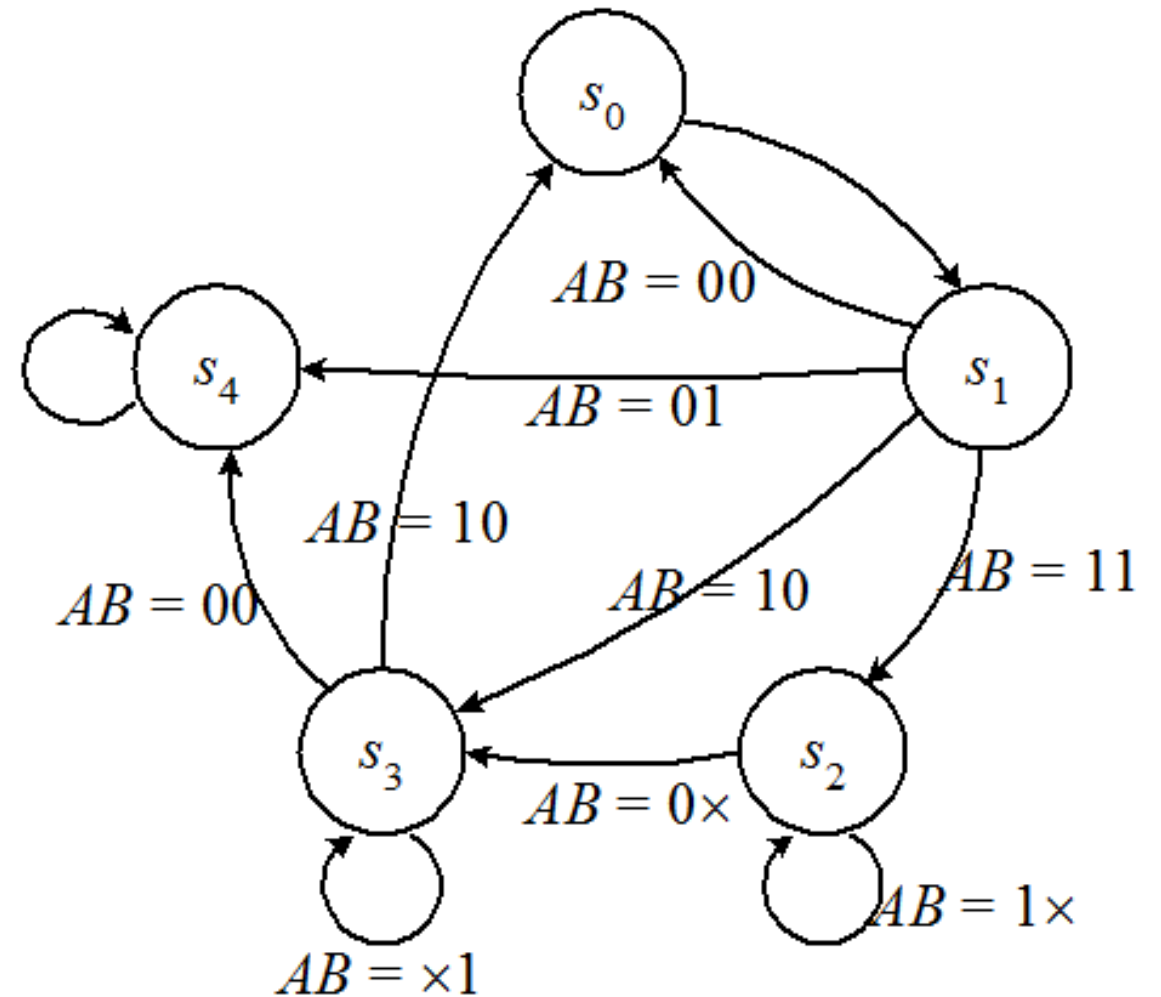
- $(C=0)$ is a conditional test input signal
- Y is an output signal



State Diagram



- A and B are inputs
- X denotes don't care
- Unconditional edge has no input signal label

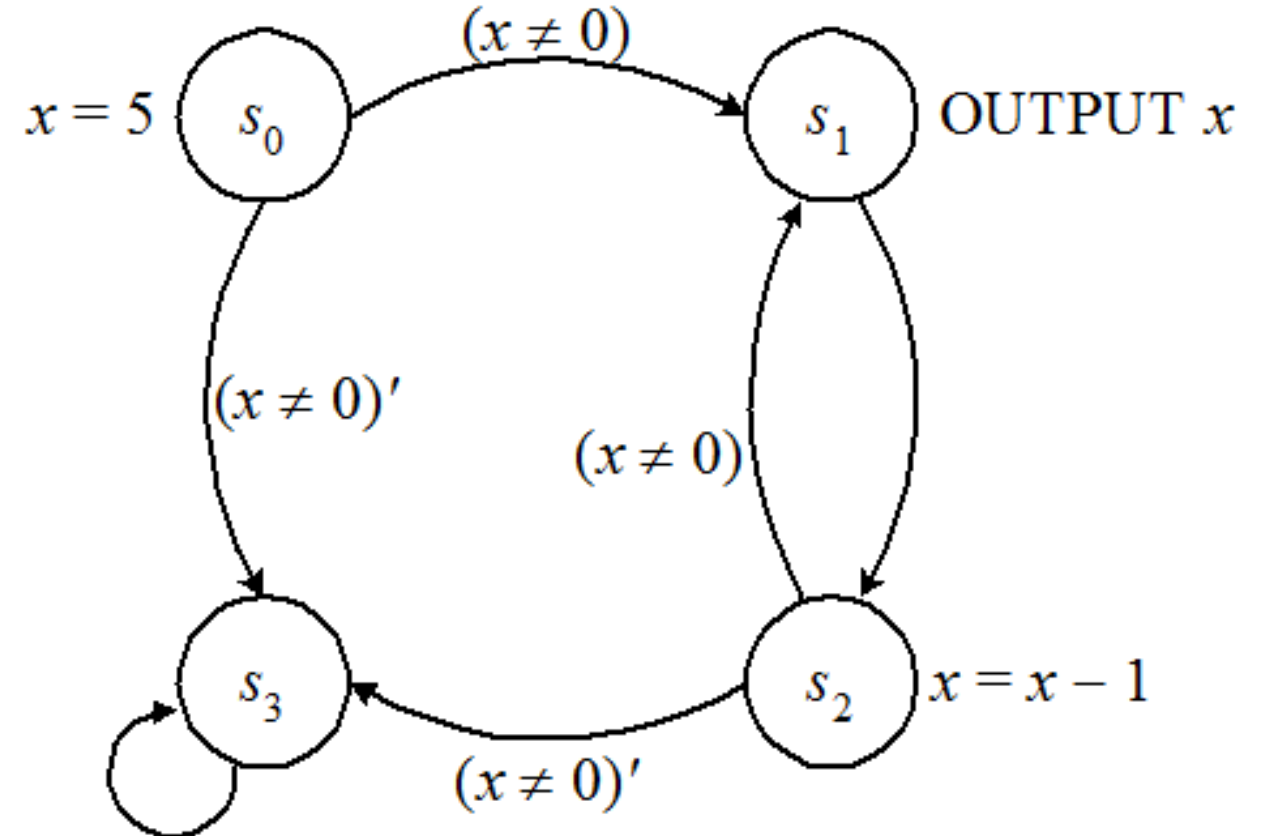


- Convert pseudo-code to a state diagram
- Data manipulations and operations are assigned to nodes
- Instruction sequencing and conditional tests are assigned to edges

Creating State Diagrams

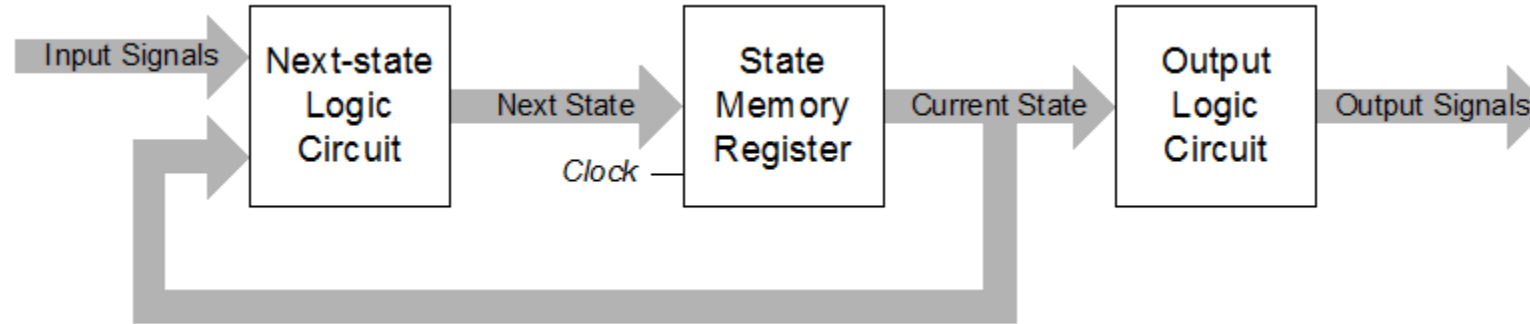


```
x = 5  
WHILE (x ≠ 0) {  
  OUTPUT x  
  x = x - 1  
}
```

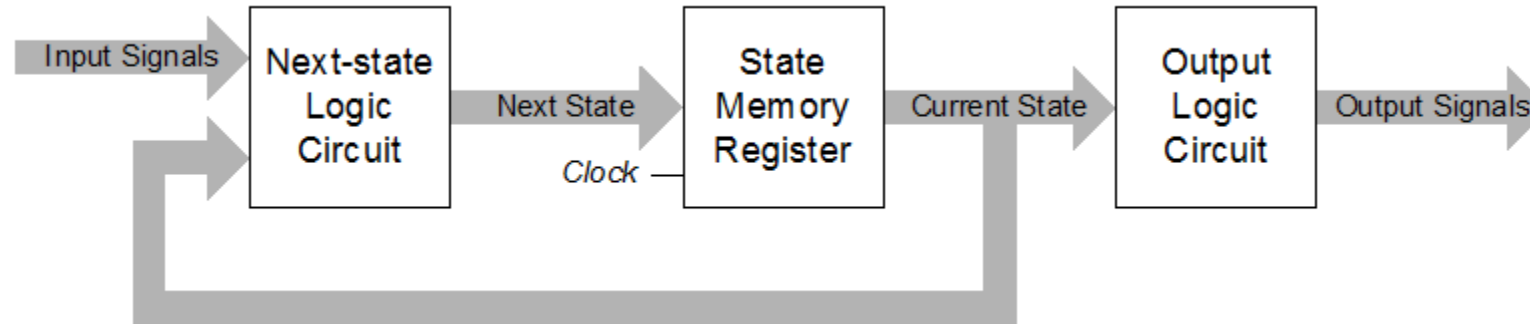




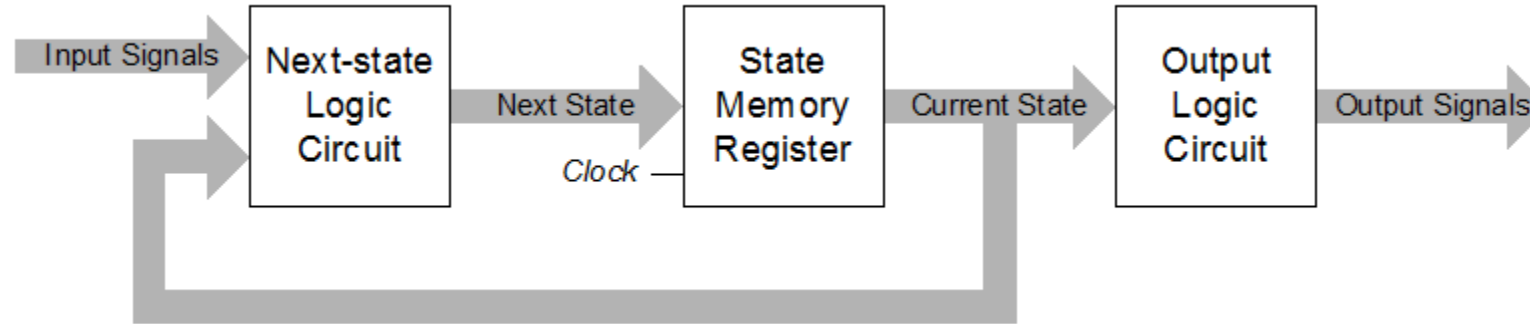
1. Need to have memory (D Flip-Flops) to remember the current state
2. Need to have a circuit to determine the next state to go to
3. Need to have a circuit to generate output signals



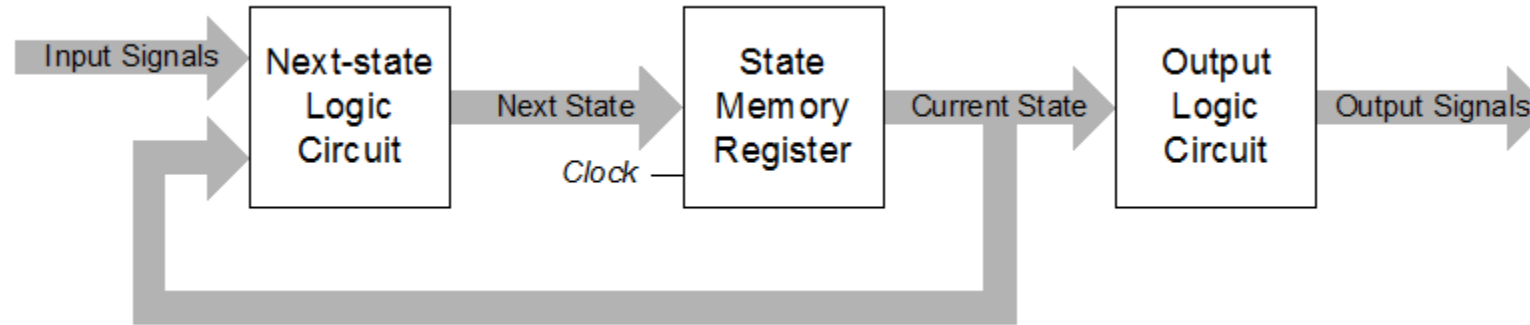
- Three components:
 1. State memory (D Flip-flops, Sequential)
 2. Next-state logic circuit (Combinational)
 3. Output logic circuit (Combinational)



- State memory :
 - Sequential circuit
 - One or more D flip-flops
 - Number of flip-flops needed depends on how many states

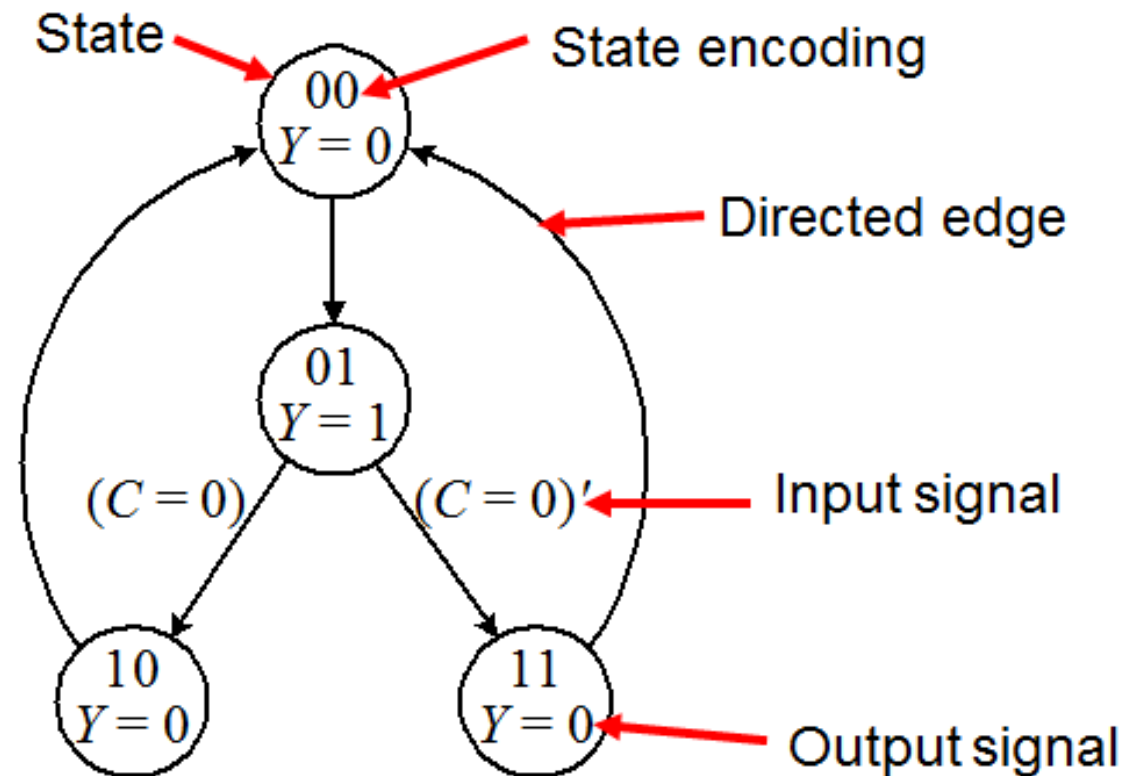


- Next-state logic circuit :
 - Combinational circuit
 - Depends on input signals and current state
 - Output next state to go to

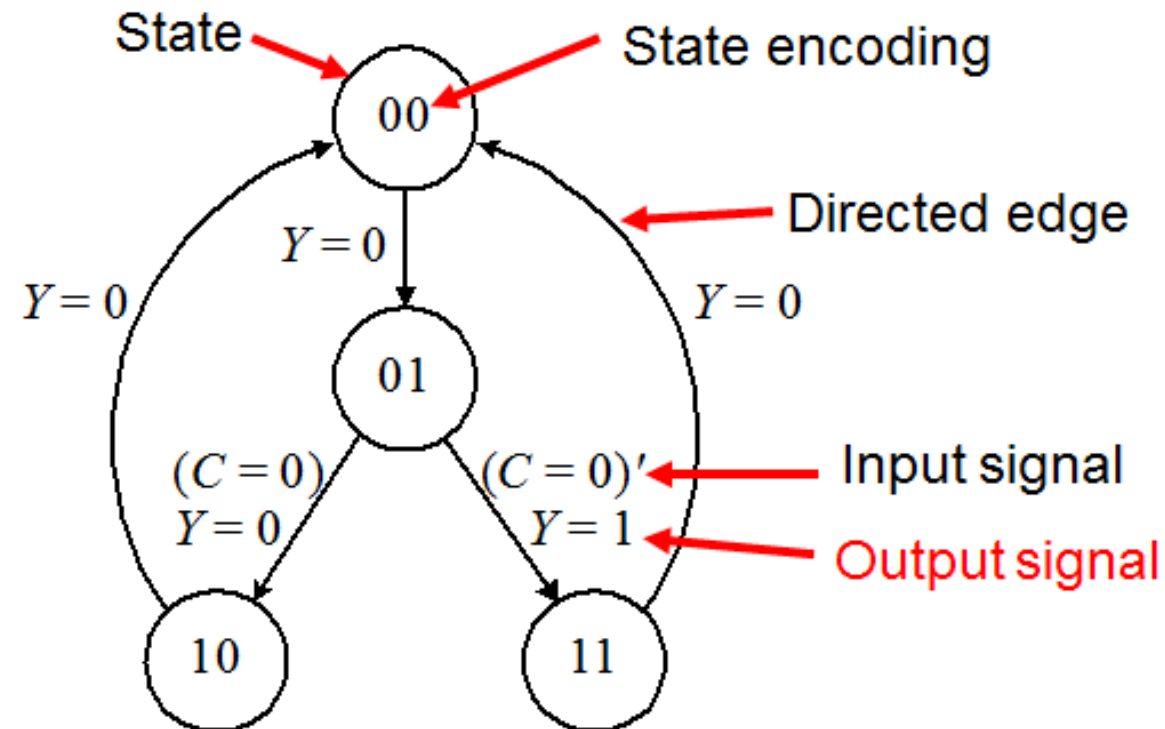


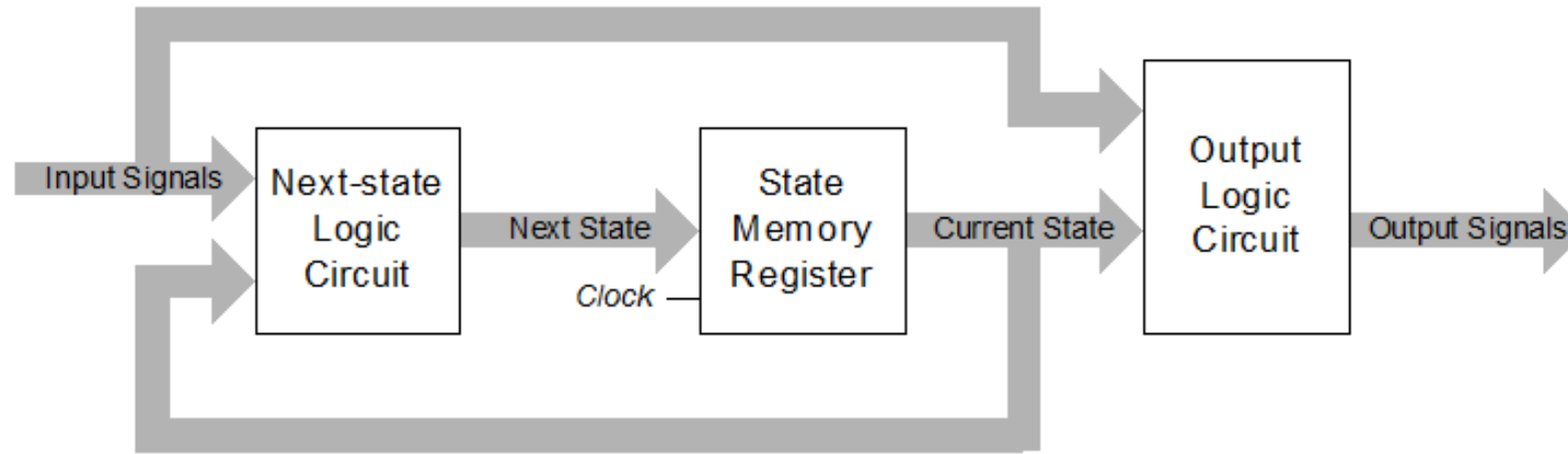
- Output logic circuit :
 - Combinational circuit
 - Depends on current state only (Moore)
 - Depends on current state and input signals (Mealy)
 - Output control signals

- Moore (output dependent only on the current state)



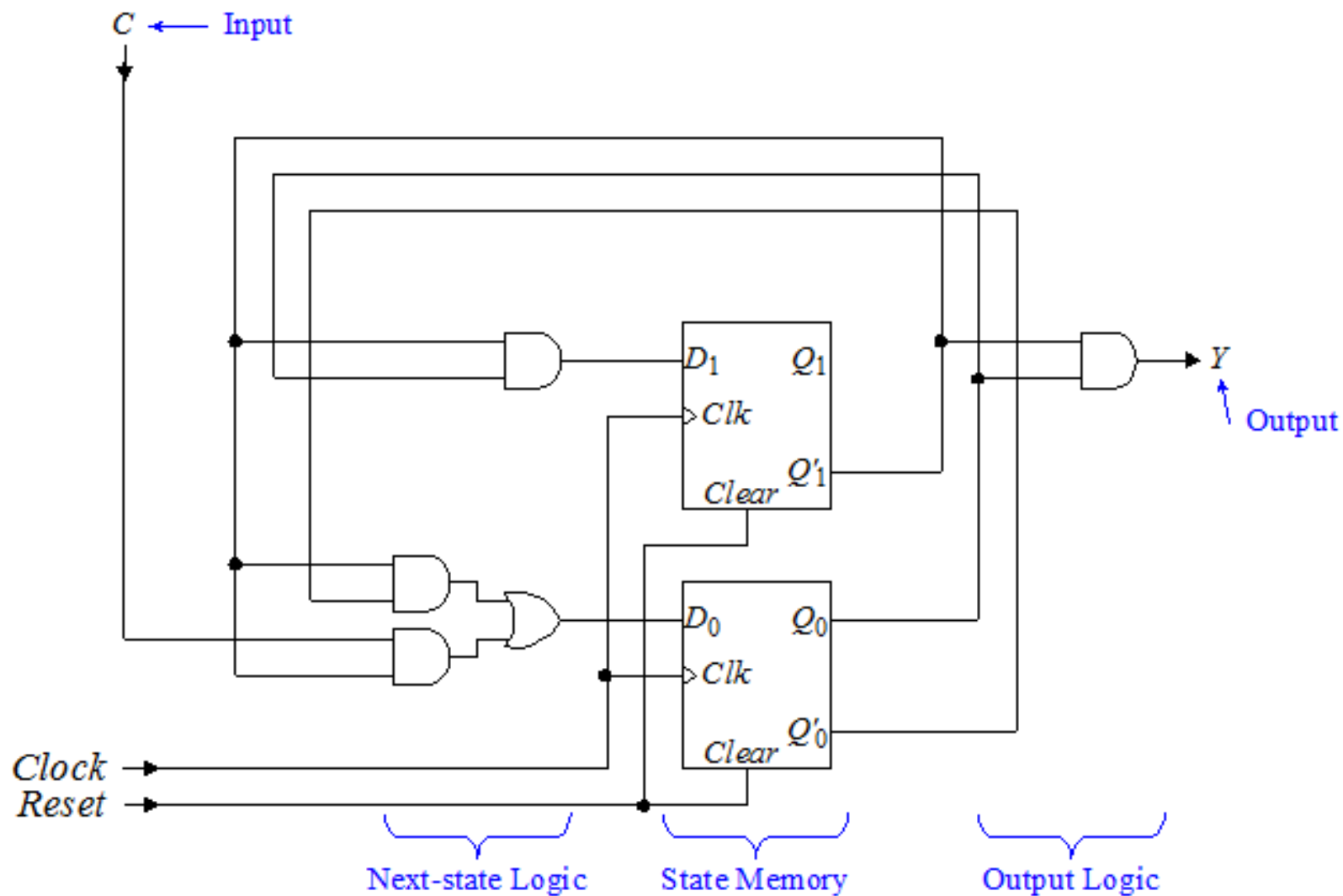
- Mealy (output dependent on the current state and the input signal)



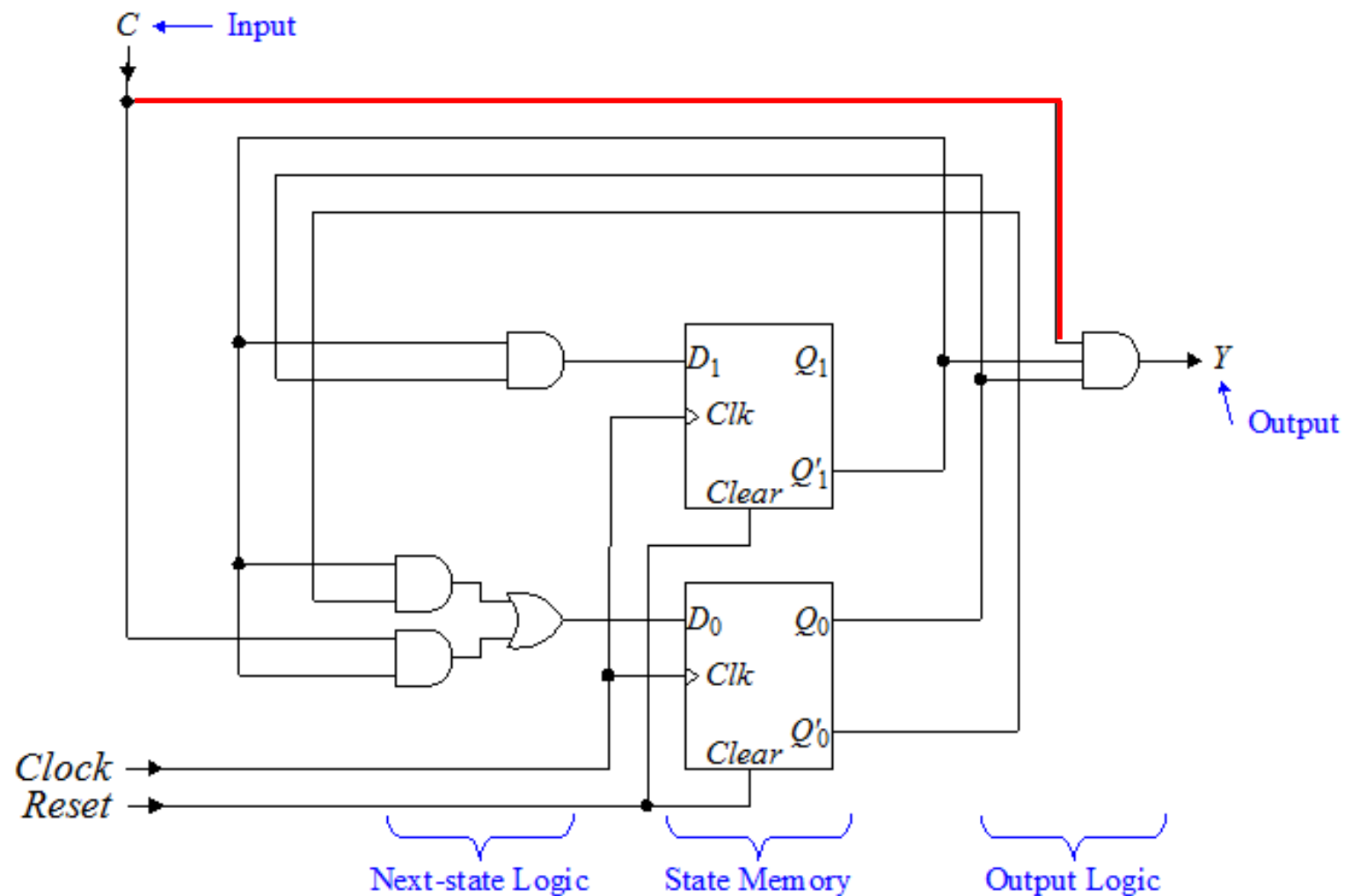


- Clock
 - The content of the state memory flip-flops will change at every rising edge of the clock signal
 - FSM will go to a new state at every clock

Sample Moore FSM Circuit



Sample Mealy FSM Circuit

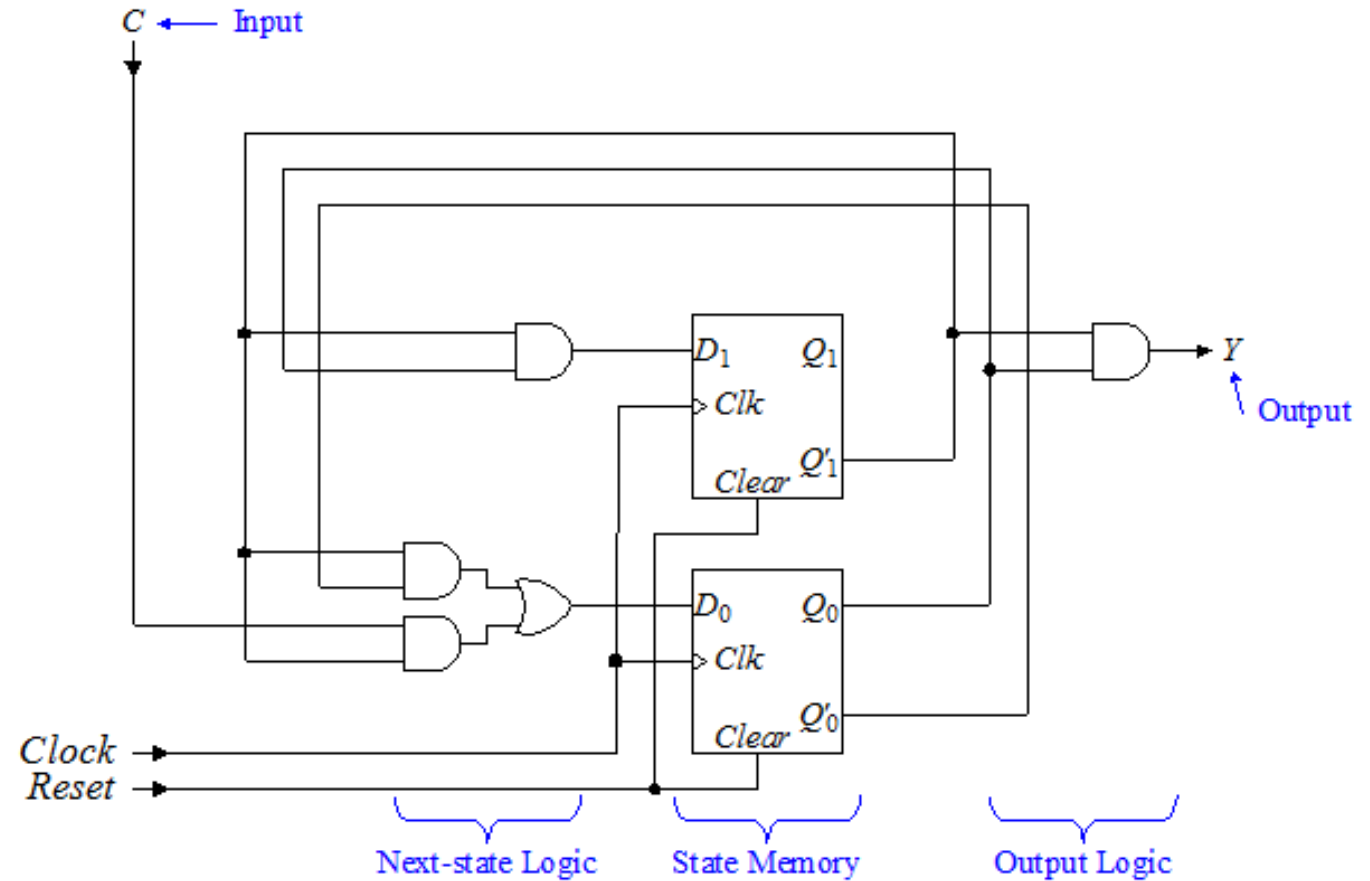


- Derive state diagram from a given FSM circuit
- Steps:
 - 1. Derive the next-state equations and truth table from the next-state logic circuit
(Analysis of a combinational circuit)**
 - 2. Derive the output equations and truth table from the output logic circuit
(Analysis of a combinational circuit)**
 - 3. Draw the state diagram based on 1 and 2**

Example



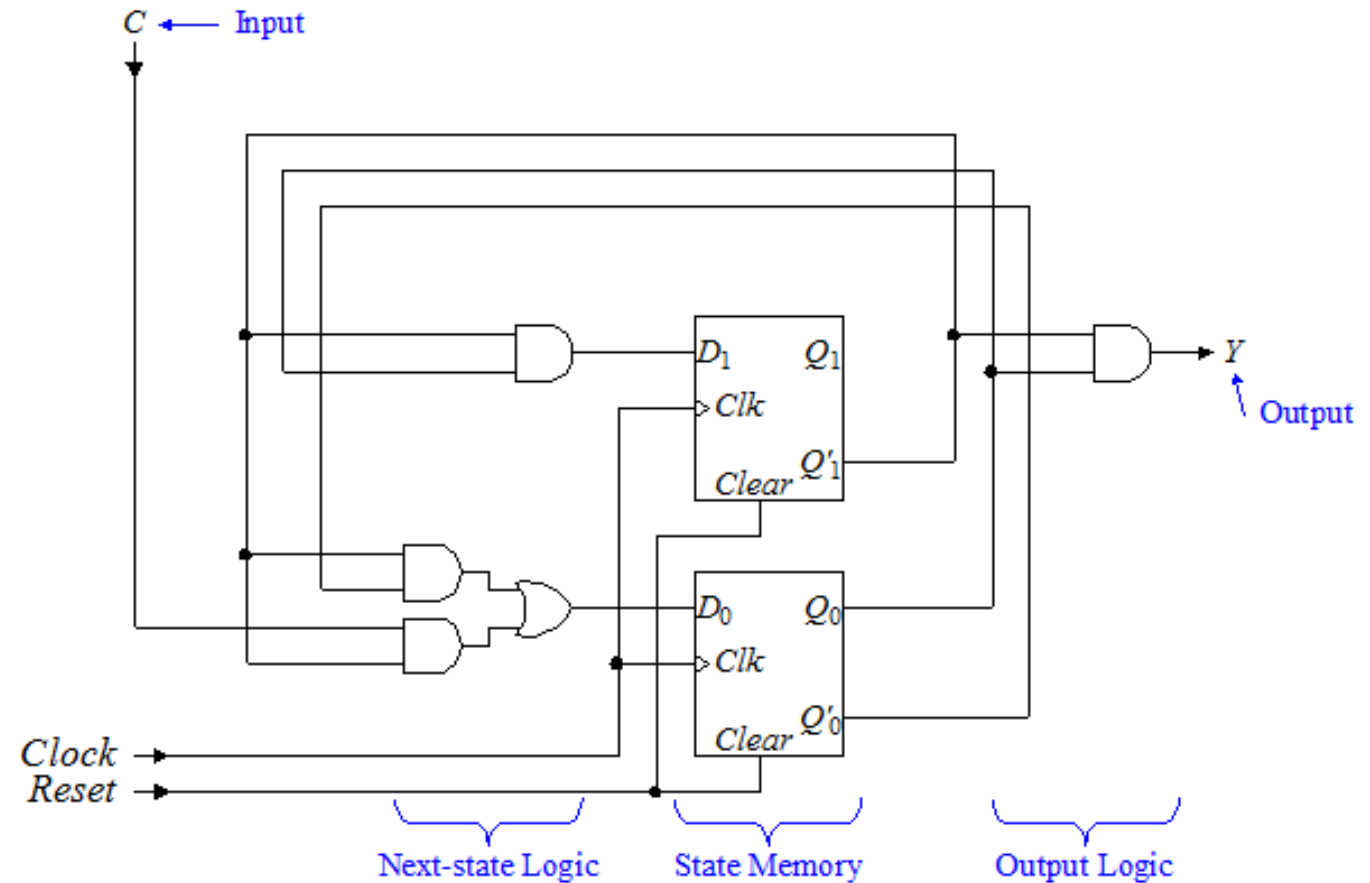
- Derive the state diagram for this FSM



Next-state Equations



$$Q_{1\text{next}} = D_1 = Q_1'Q_0$$
$$Q_{0\text{next}} = D_0 = Q_1'Q_0' + CQ_1'$$



$$Q_{1\text{next}} = D_1 = Q_1' Q_0$$
$$Q_{0\text{next}} = D_0 = Q_1' Q_0' + C Q_1'$$

CQ_1Q_0	$Q_{1\text{next}} Q_{0\text{next}}$
000	01
001	10
010	00
011	00
100	01
101	11
110	00
111	00

Next-state Table



$$Q_{1\text{next}} = D_1 = Q_1'Q_0$$
$$Q_{0\text{next}} = D_0 = Q_1'Q_0' + CQ_1'$$

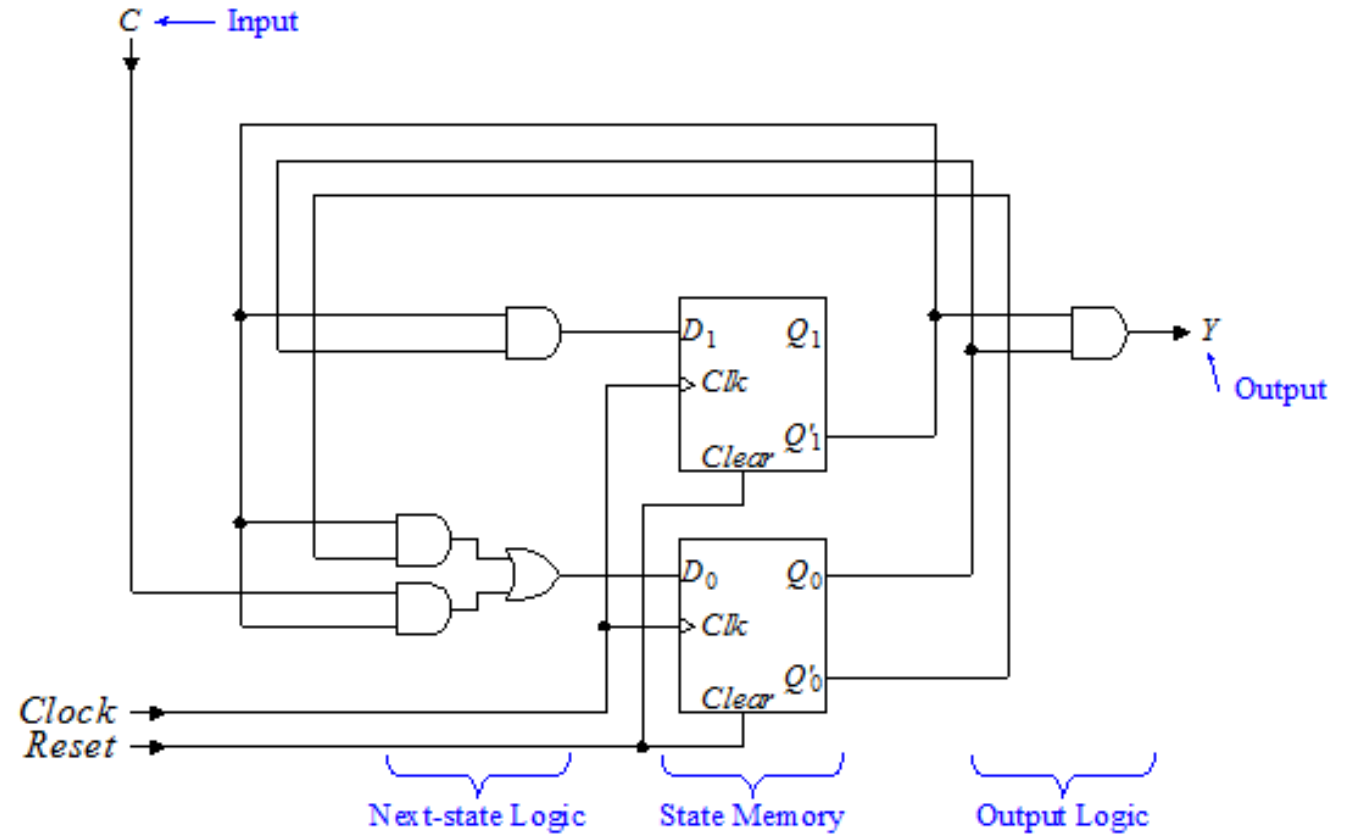
Current State Q_1Q_0	Next State	
	$Q_{1\text{next}}Q_{0\text{next}}$	
	$C = 0$	$C = 1$
00	01	01
01	10	11
10	00	00
11	00	00

Output Equation and Table



$$Y = Q_1' Q_0$$

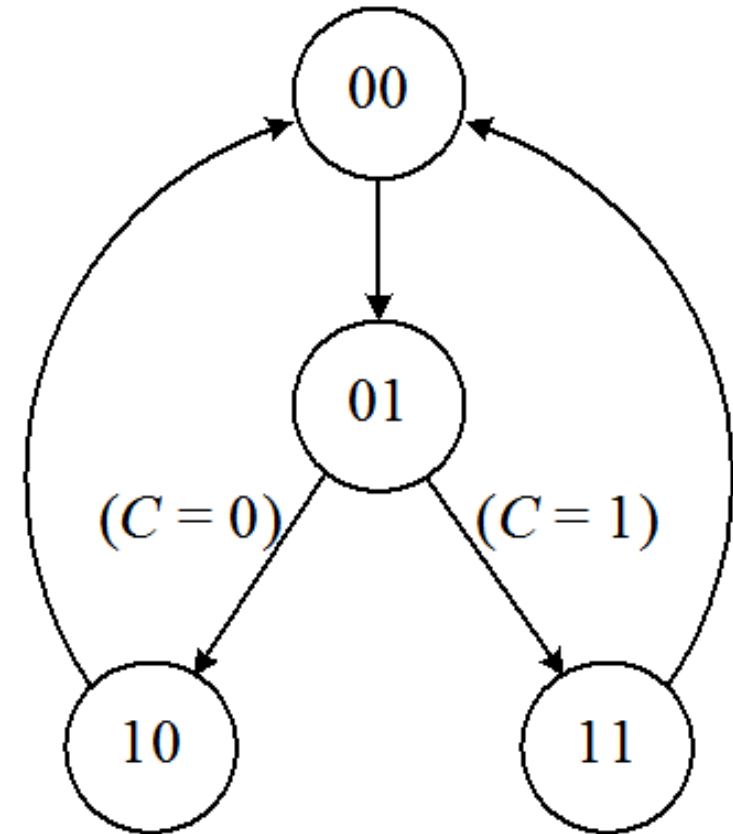
$Q_1 Q_0$	Y
00	0
01	1
10	0
11	0



State Diagram



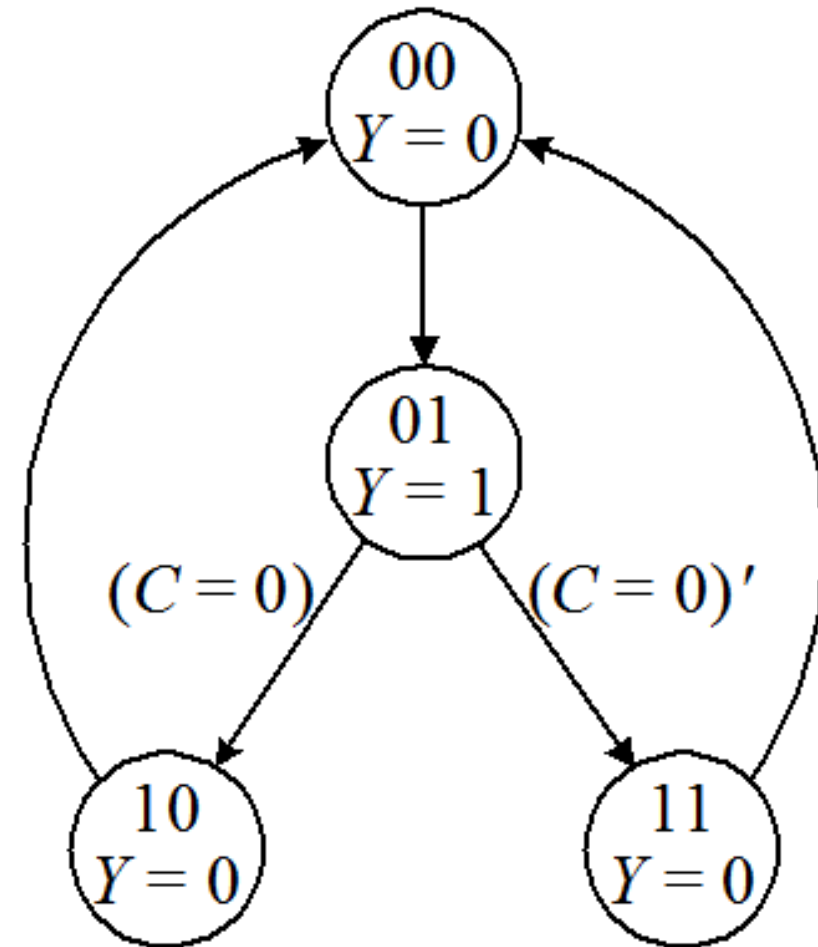
Current State Q_1Q_0	Next State	
	$Q_{1\text{next}}$ $C = 0$	$Q_{0\text{next}}$ $C = 1$
00	01	01
01	10	11
10	00	00
11	00	00



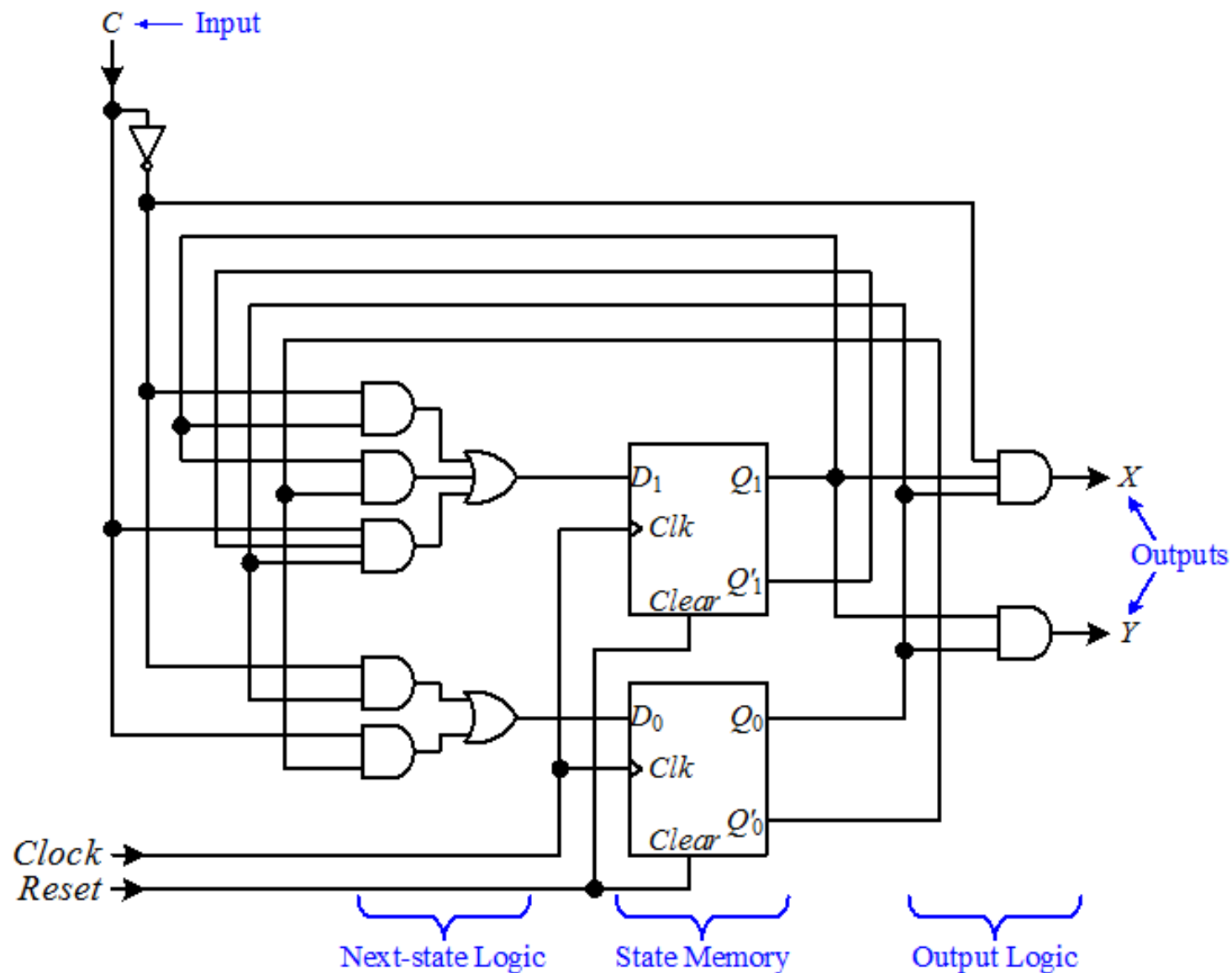
State Diagram



Q_1Q_0	Y
00	0
01	1
10	0
11	0



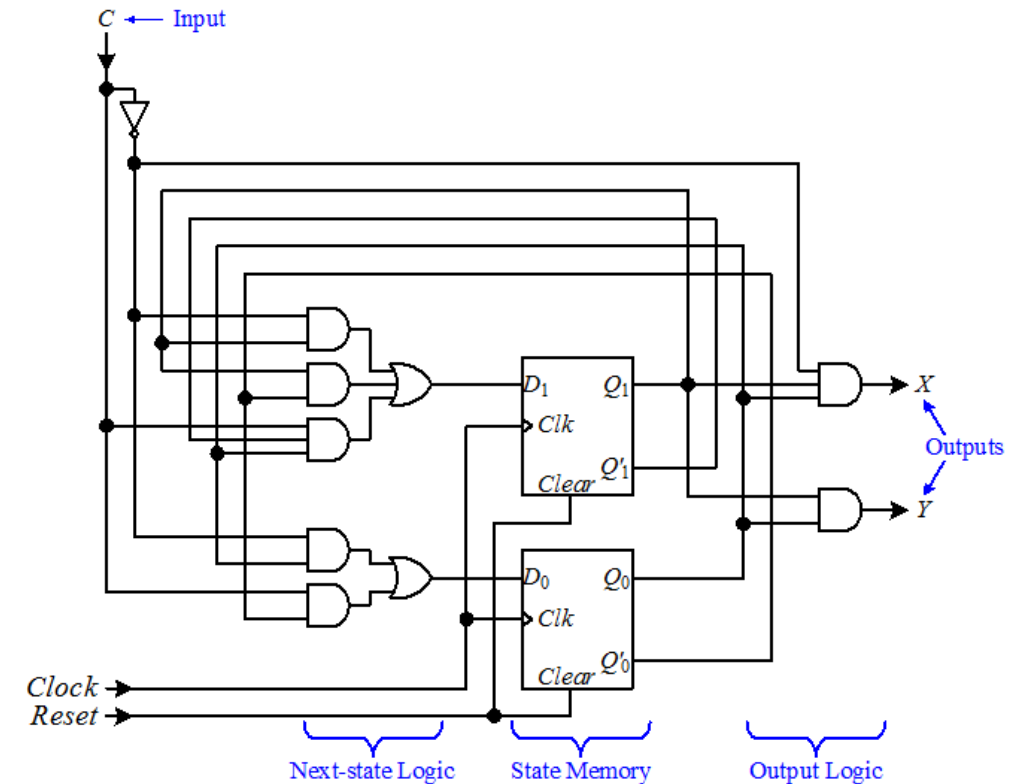
Example 2 – Mealy FSM



Next-state Equations



$$Q_{1\text{next}} = D_1 = C'Q_1 + Q_1Q_0' + CQ_1'Q_0$$
$$Q_{0\text{next}} = D_0 = C'Q_0 + CQ_0'$$



Next-state Equations and Table



$$Q_{1\text{next}} = D_1 = C'Q_1 + Q_1Q_0' + CQ_1'Q_0$$
$$Q_{0\text{next}} = D_0 = C'Q_0 + CQ_0'$$

Current State Q_1Q_0	Next State D_1D_0	
	$C = 0$	$C = 1$
00	00	01
01	01	10
10	10	11
11	11	00

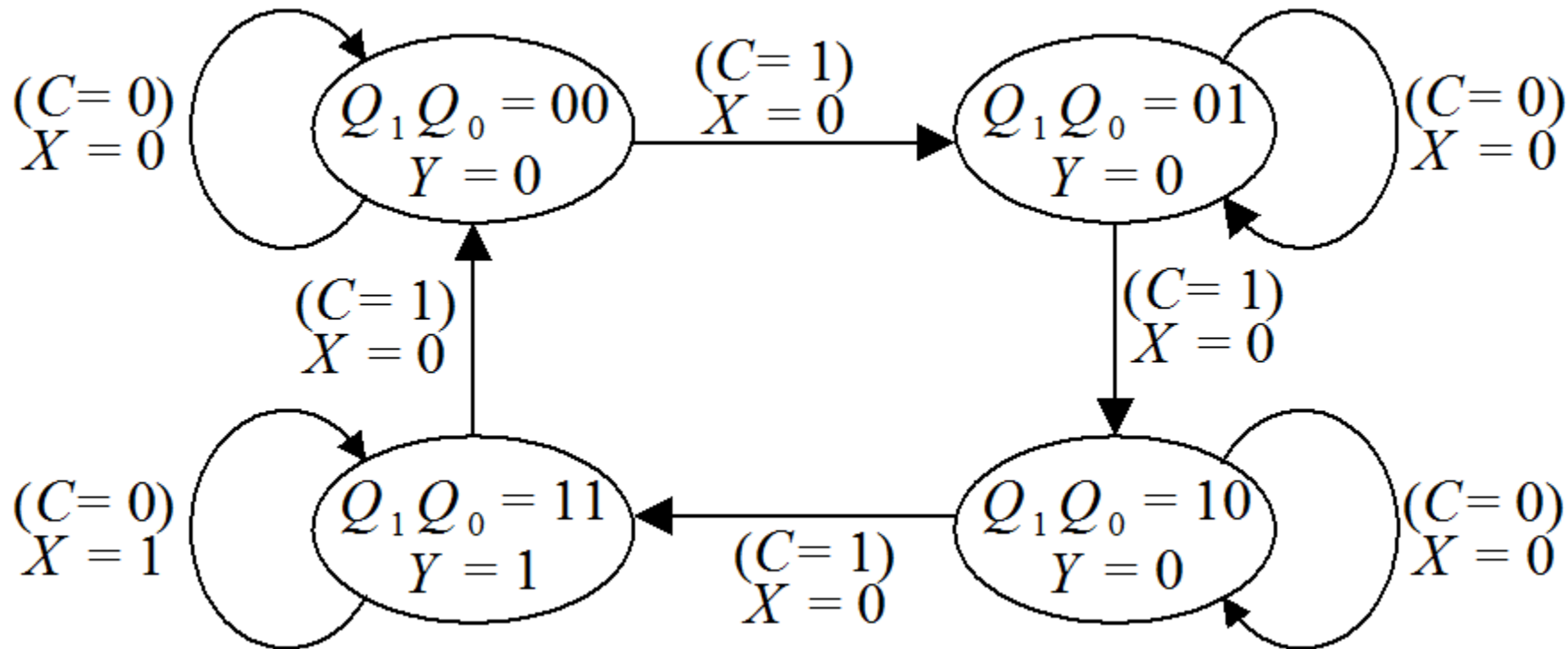
Output Equation and Table



$$X = C'Q_1Q_0$$
$$Y = Q_1Q_0$$

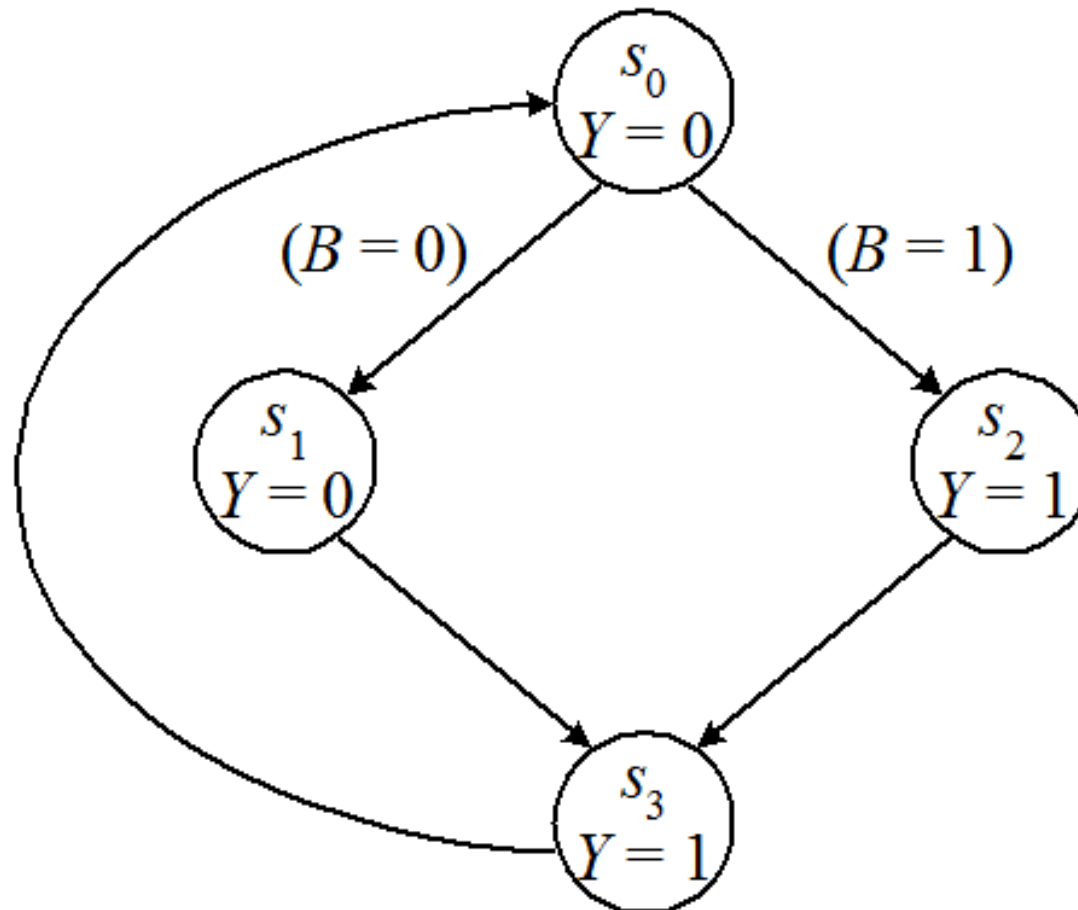
Current State Q_1Q_0	Output X		Output Y
	$C = 0$	$C = 1$	
00	0	0	0
01	0	0	0
10	0	0	0
11	1	0	1

State Diagram



- Derive FSM circuit from a given state diagram
- Steps:
 - 1. Derive the next-state table and equations from the state diagram**
 - 2. Derive the output table and equations from the state diagram**
 - 3. Draw the FSM circuit based on the equations from 1 and 2**

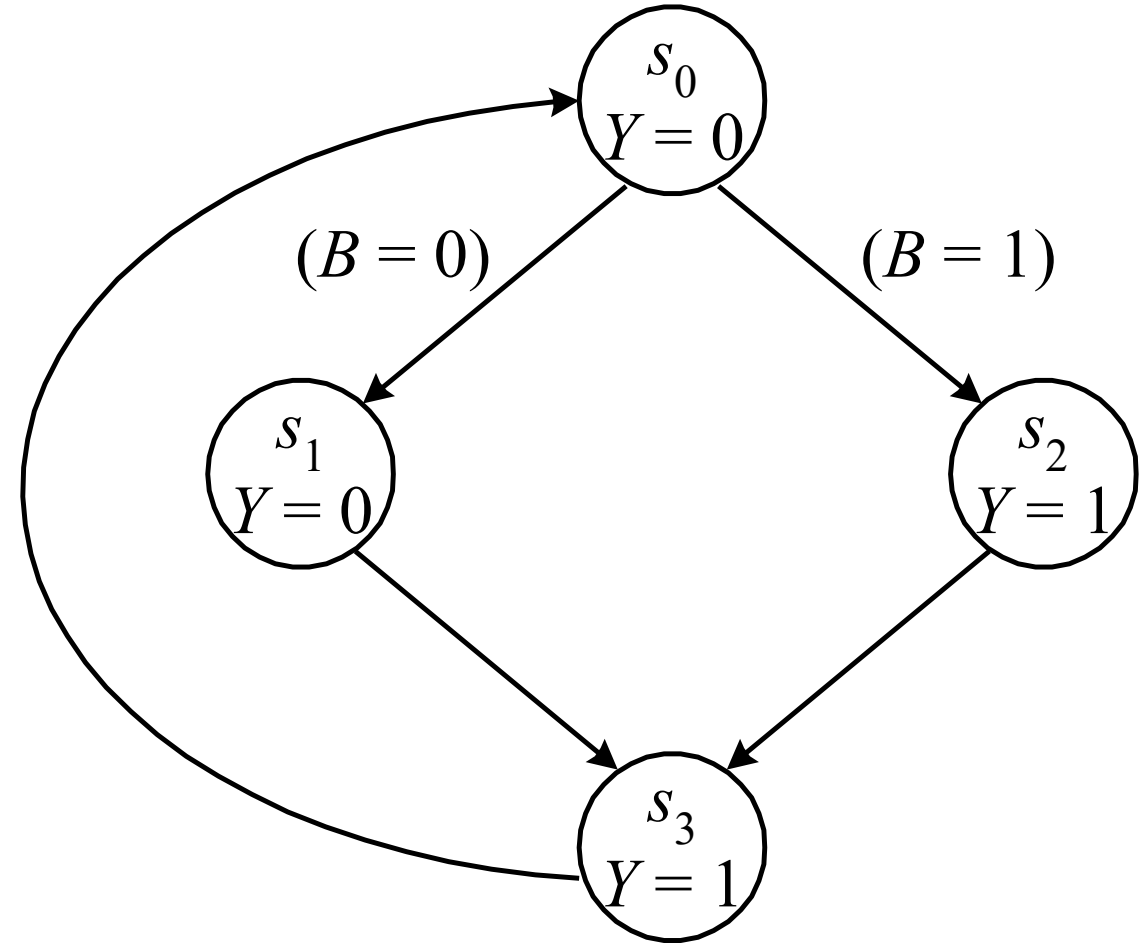
- Derive the FSM for this state diagram



Next-state Table



Current State Q_1Q_0	Next State D_1D_0	
	$B = 0$	$B = 1$
s_0 00	s_1 01	s_2 10
s_1 01	s_3 11	s_3 11
s_2 10	s_3 11	s_3 11
s_3 11	s_0 00	s_0 00



Next-state Equations



Current State Q_1Q_0	Next State $Q_{1next} Q_{0next} (D_1D_0)$	
	$B = 0$	$B = 1$
s_0 00	s_1 01	s_2 10
s_1 01	s_3 11	s_3 11
s_2 10	s_3 11	s_3 11
s_3 11	s_0 00	s_0 00

$$Q_{1next} = D_1 = BQ_1'Q_0' + Q_1'Q_0 + Q_1Q_0'$$
$$Q_{0next} = D_0 = B'Q_1'Q_0' + Q_1'Q_0 + Q_1Q_0'$$

Next-state Equations



D_1 B Q_1Q_0		0	1	
	00		1	BQ_1'
	01	1	1	$Q_1'Q_0$
	11			
	10	1	1	Q_1Q_0'

D_0 B Q_1Q_0		0	1	
	00	1		$B'Q_1'$
	01	1	1	$Q_1'Q_0$
	11			
	10	1	1	Q_1Q_0'

$$Q_{1\text{next}} = D_1 = BQ_1'Q_0' + Q_1'Q_0 + Q_1Q_0'$$

$$= BQ_1' + (Q_1 \oplus Q_0)$$

$$Q_{0\text{next}} = D_0 = B'Q_1'Q_0' + Q_1'Q_0 + Q_1Q_0'$$

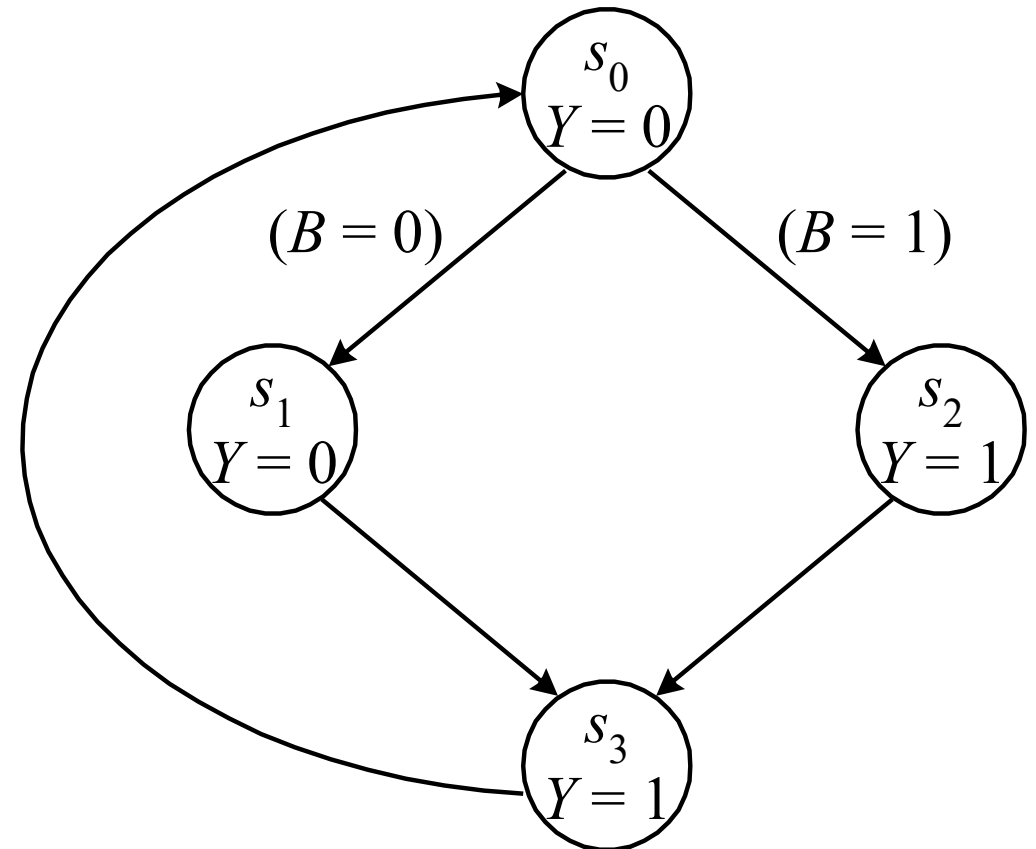
$$= B'Q_1' + (Q_1 \oplus Q_0)$$

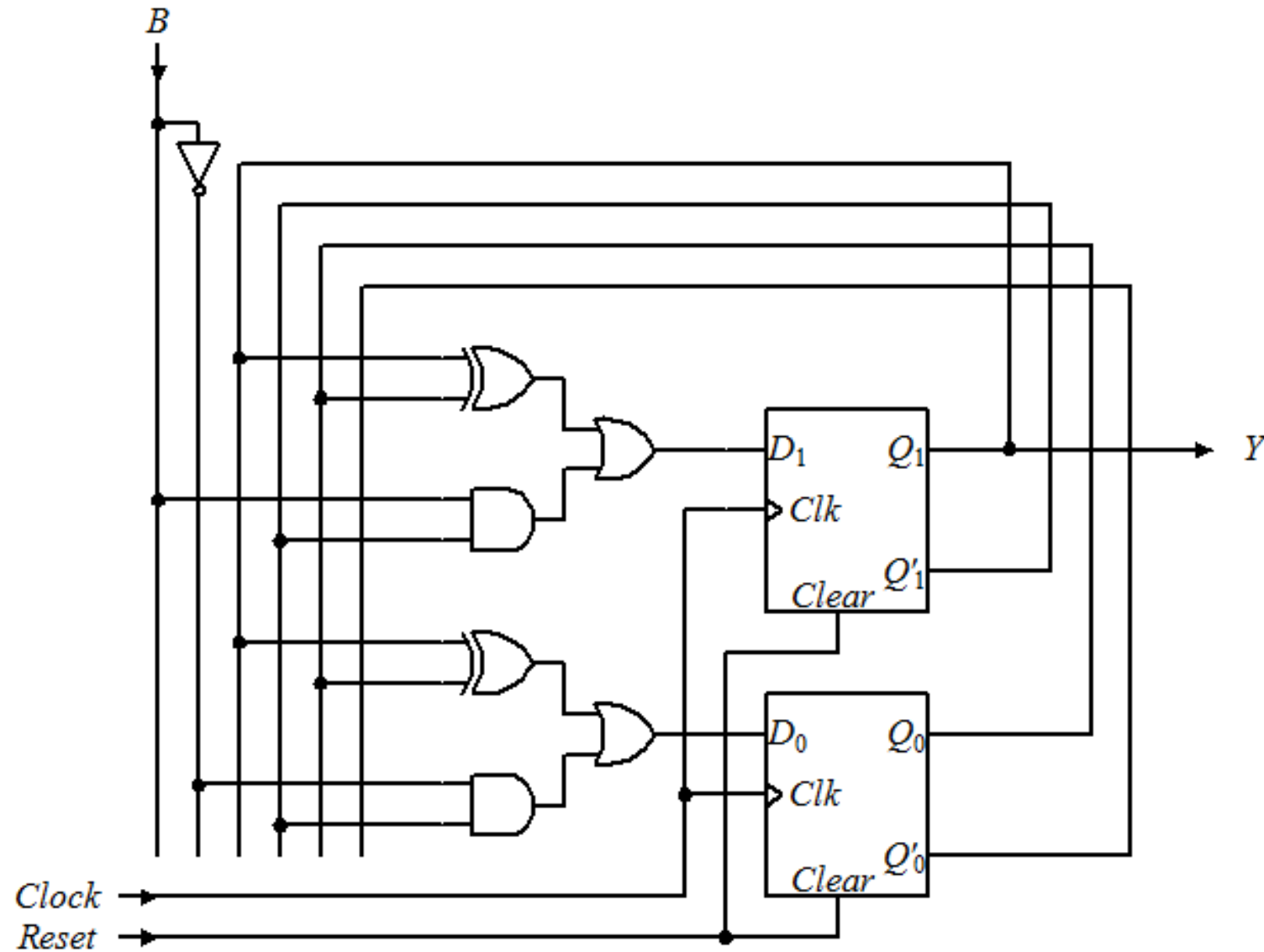
Output Table and Equation



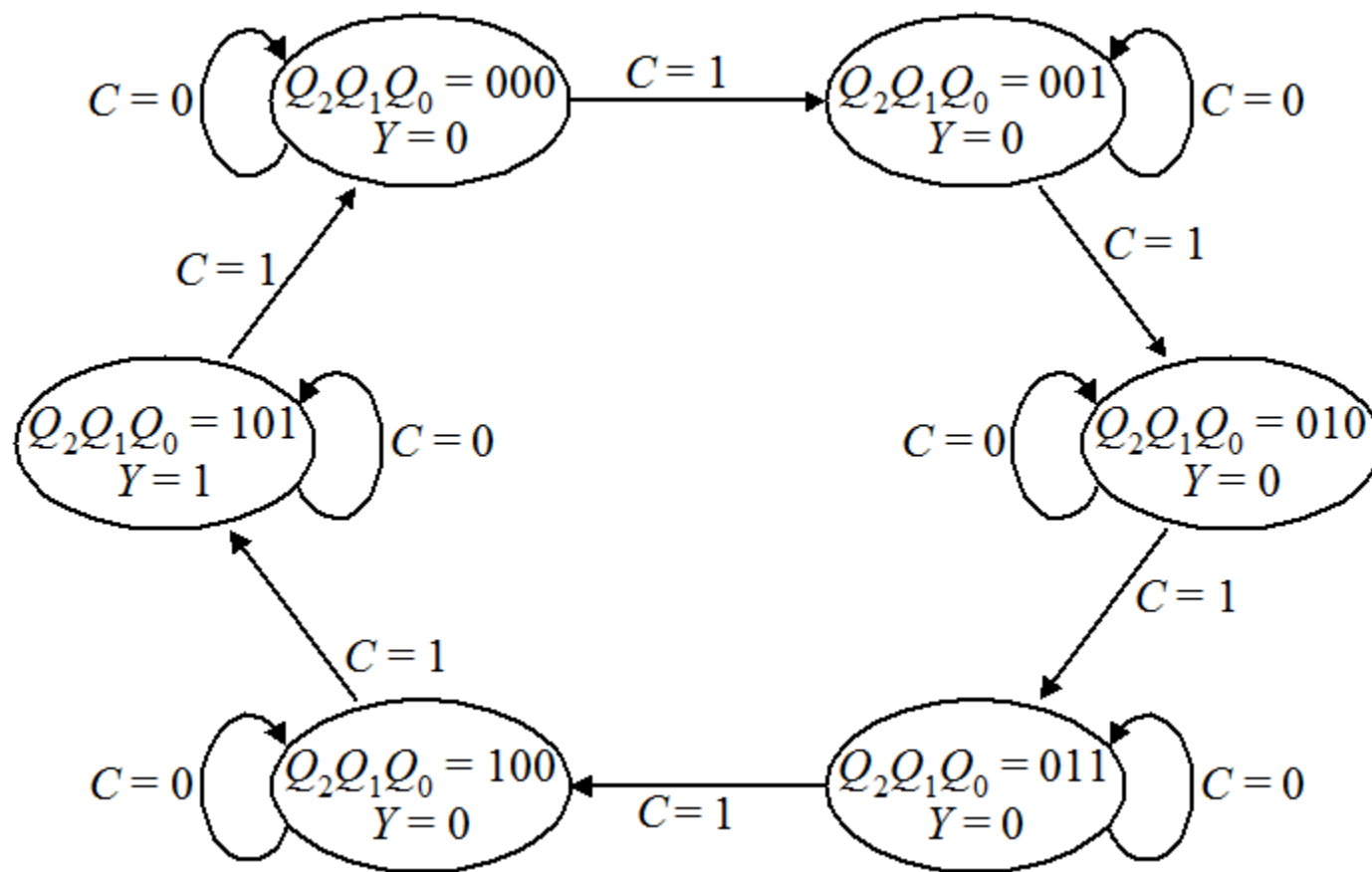
Current State Q_1Q_0	Output Y
s_0 00	0
s_1 01	0
s_2 10	1
s_3 11	1

$$Y = Q_1$$





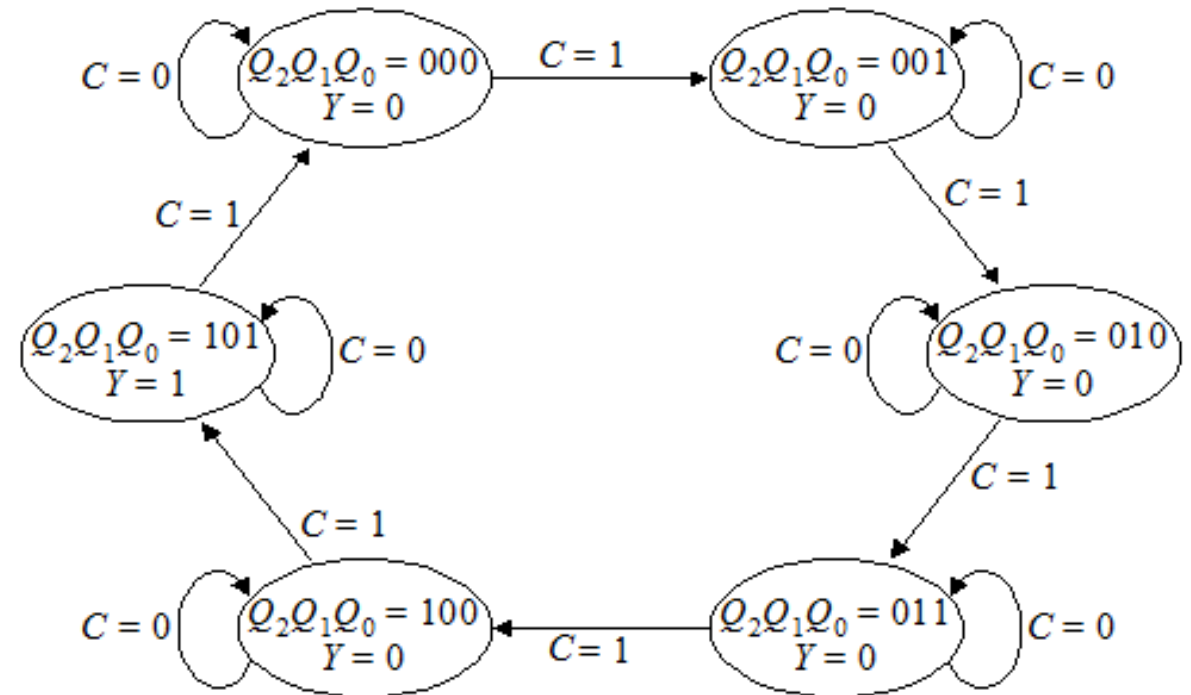
Example 2



Next-state Table



Current State $Q_2Q_1Q_0$	Next State	
	$Q_{2next} Q_{1next} Q_{0next}$	
	$C = 0$	$C = 1$
000	000	001
001	001	010
010	010	011
011	011	100
100	100	101
101	101	000



Next-state Equations



Current State $Q_2Q_1Q_0$	Next State	
	$Q_{2next} Q_{1next} Q_{0next}$	
	$C = 0$	$C = 1$
000	000	001
001	001	010
010	010	011
011	011	100
100	100	101
101	101	000

$$Q_{2next} = C'Q_2 + Q_2Q_0' + CQ_1Q_0$$

$$Q_{1next} = C'Q_1 + Q_1Q_0' + CQ_2'Q_1'Q_0$$

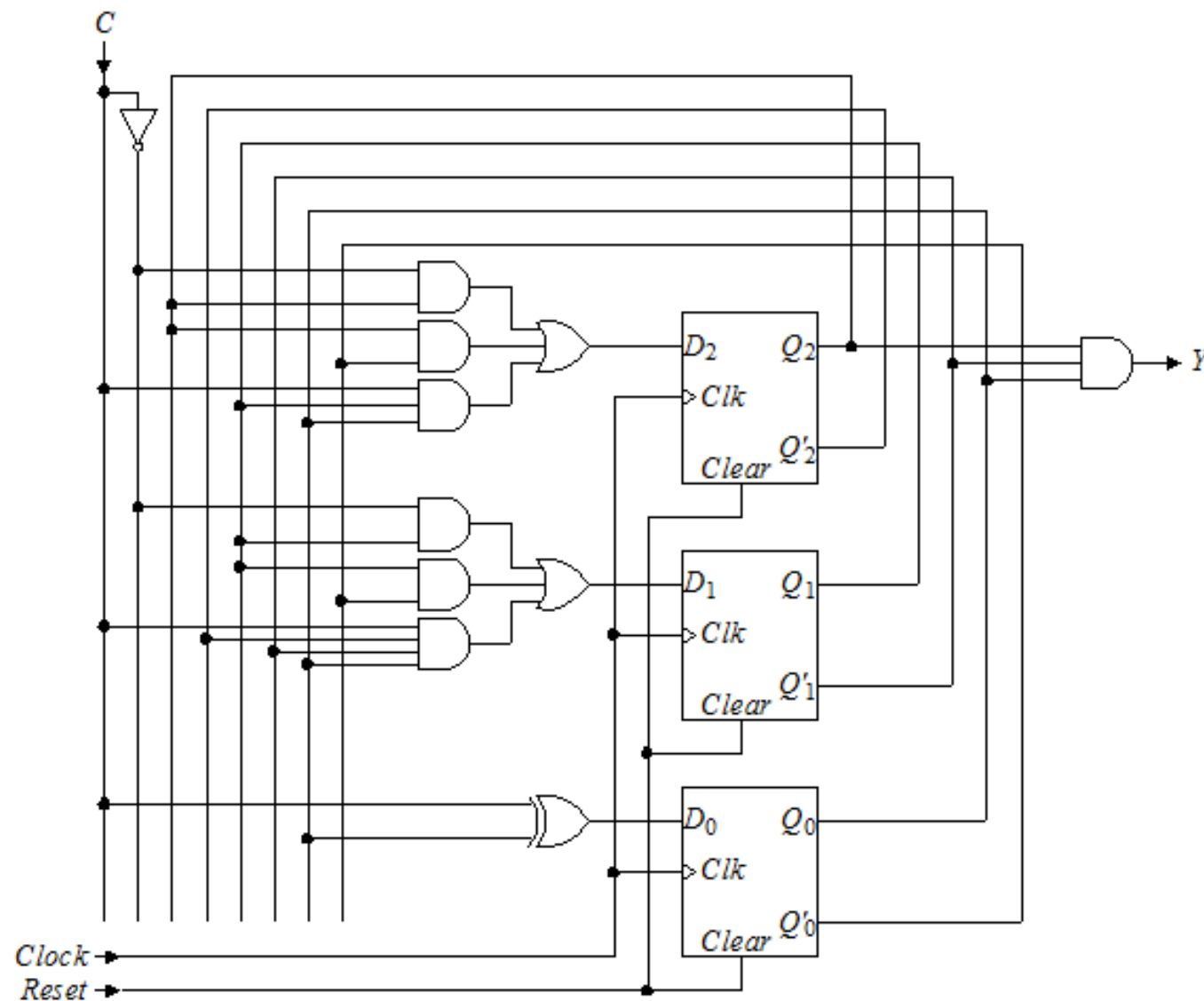
$$Q_{0next} = C'Q_0 + CQ_0' = C \oplus Q_0$$

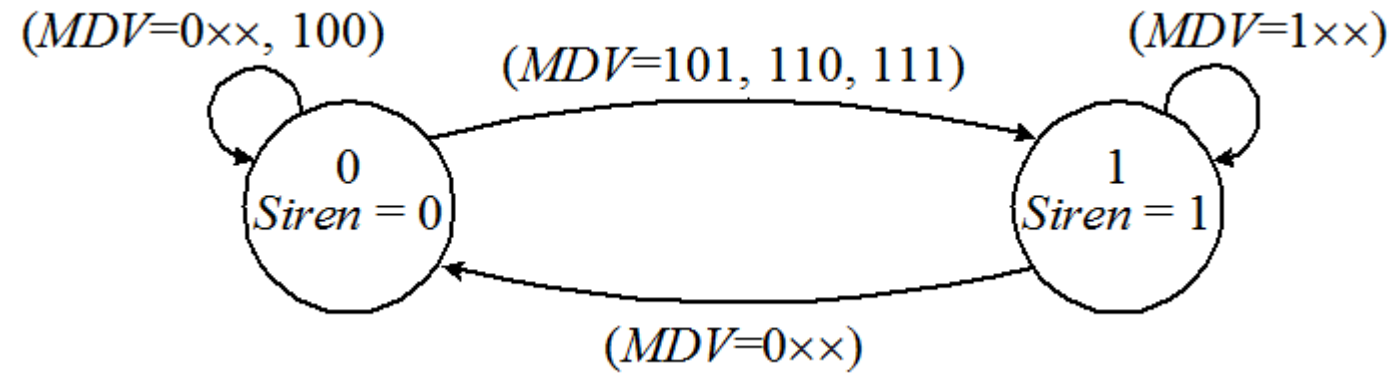
Output Table and Equation



Current State $Q_2Q_1Q_0$	Output Y
000	0
001	0
010	0
011	0
100	0
101	1

$$Y = Q_2Q_1'Q_0$$



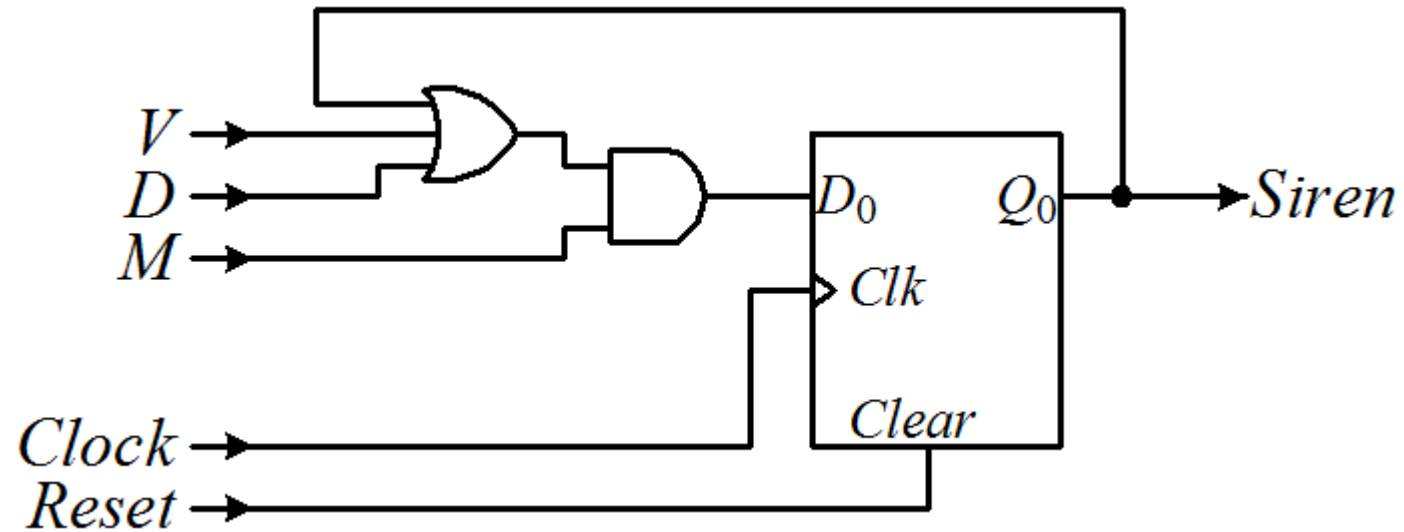


Current State Q_0	Next State							
	Q_{0next}							
	M, D, V							
	000	001	010	011	100	101	110	111
0	0	0	0	0	0	1	1	1
1	0	0	0	0	1	1	1	1

Current State Q_0	Next State Q_{0next}							
	M, D, V							
	000	001	010	011	100	101	110	111
0	0	0	0	0	0	1	1	1
1	0	0	0	0	1	1	1	1

		DV			
		00	01	11	10
MQ_0	00				
	01				
	11	1	1	1	1
	10		1	1	1

$$\begin{aligned}
 Q_{0next} &= MQ_0 + MV + MD \\
 &= M(Q_0 + V + D)
 \end{aligned}$$



$$\begin{aligned} Q_{0next} &= D_0 = MQ_0 + MV + MD \\ &= M(Q_0 + V + D) \end{aligned}$$

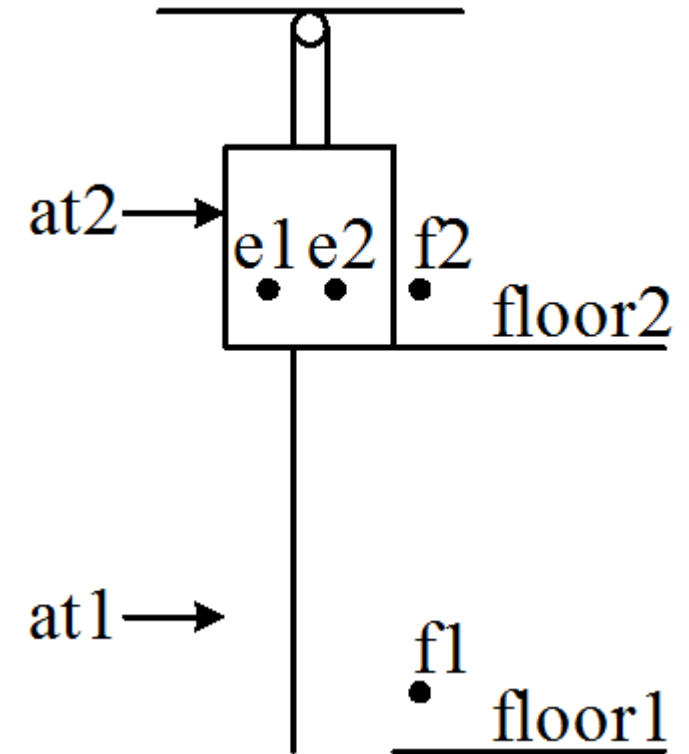
A 'Real' Example - Elevator Controller



Problem:

Design a controller for an elevator for a building with two floors.

What are the inputs and outputs?



Inputs:

- $f_1, f_2, \dots$: Buttons at each floor to call the elevator. f_1 is on floor 1, f_2 is on floor 2, etc.
- $e_1, e_2, \dots$: Buttons inside the elevator to tell the elevator which floor to go to.
- at_{1-2} : Signals from the elevator to say which floor the elevator is currently at, 0=no, 1=yes, etc.

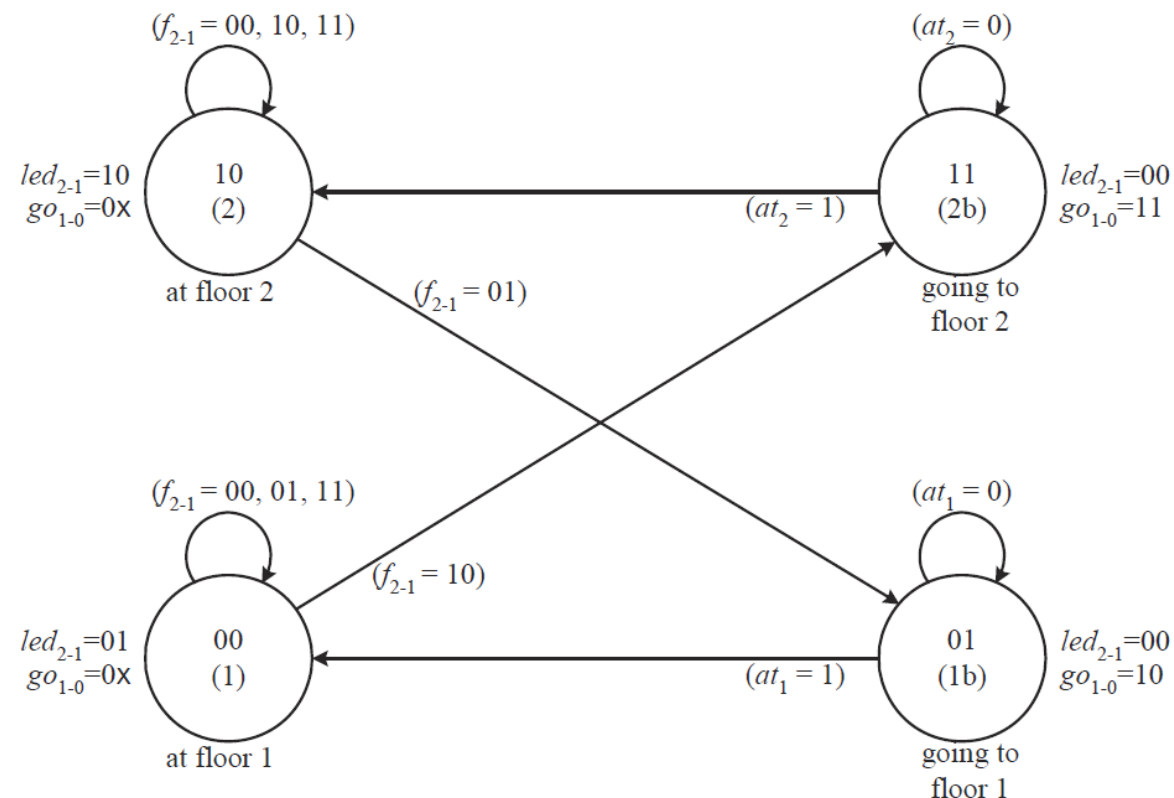
Outputs:

- go_0 : Signals to tell the elevator which floor to go to
0=floor 1, 1=floor 2, etc.
- go_1 : the most significant bit starts or stops the elevator
- led_1, led_2 : x2 LEDs on each floor to show which floor the elevator is currently at.

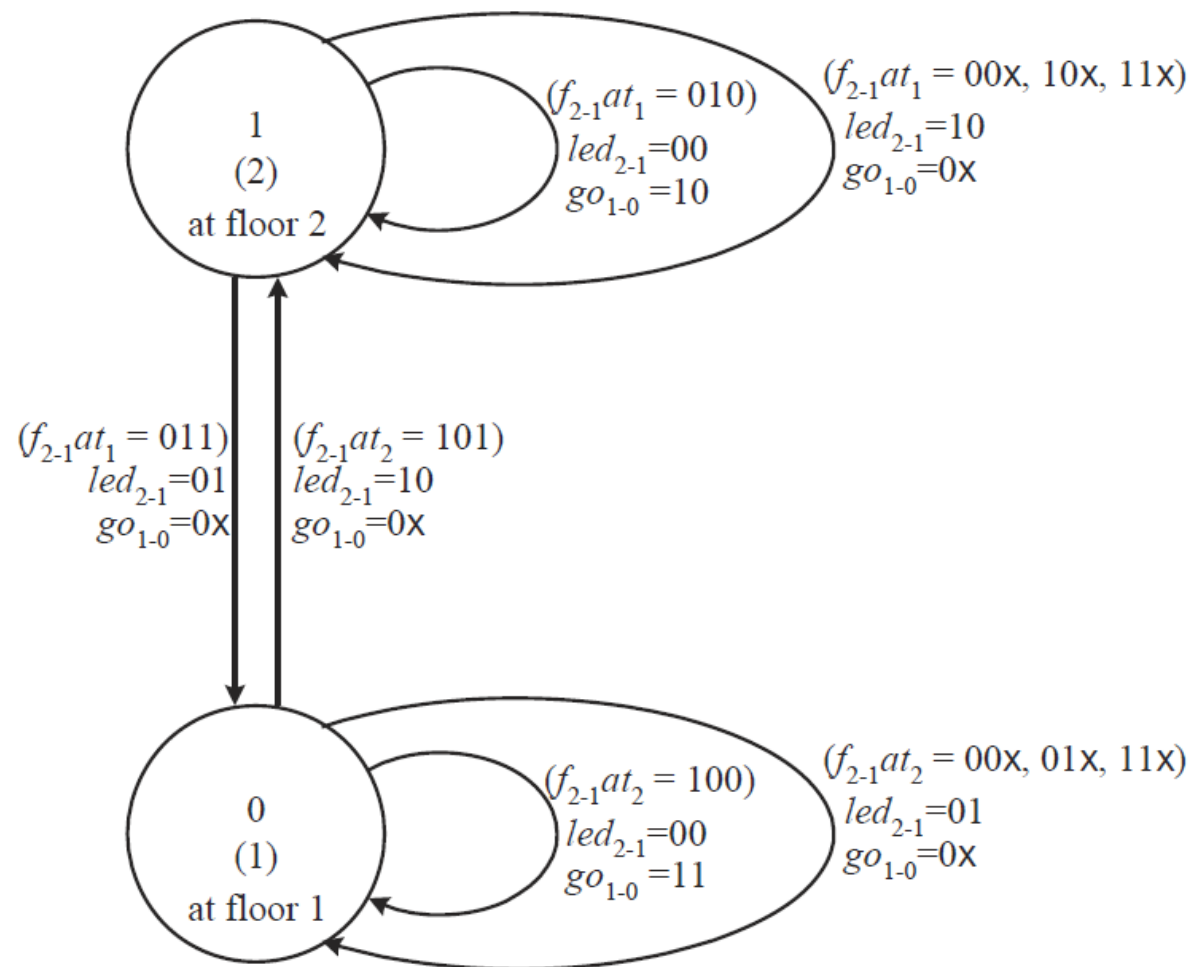
State diagram:

- What should the states represent?
 - Status of the button pressing?
 - The floor that the elevator is currently at?

States represent the floor that the elevator is currently at

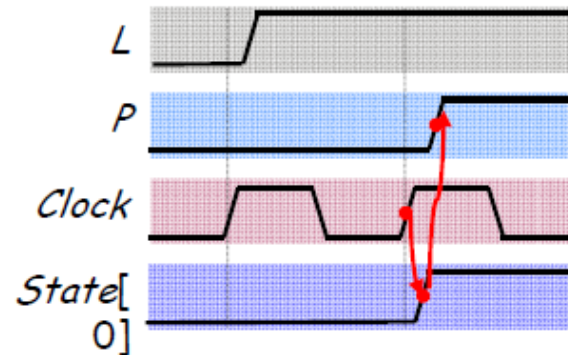


Elevator Controller with Moore FSM

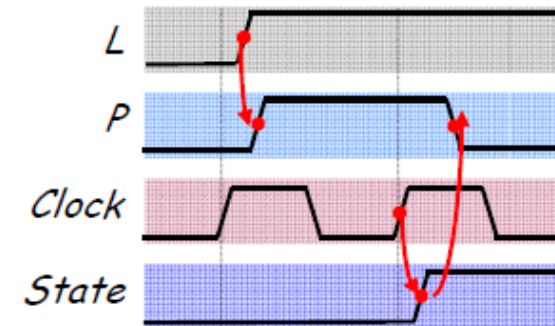


- How are they different?
 - Moore: **outputs** = $f(\text{state})$ only
 - Mealy **outputs** = $f(\text{state and input})$
 - Mealy outputs generally occur one cycle earlier than a Moore:

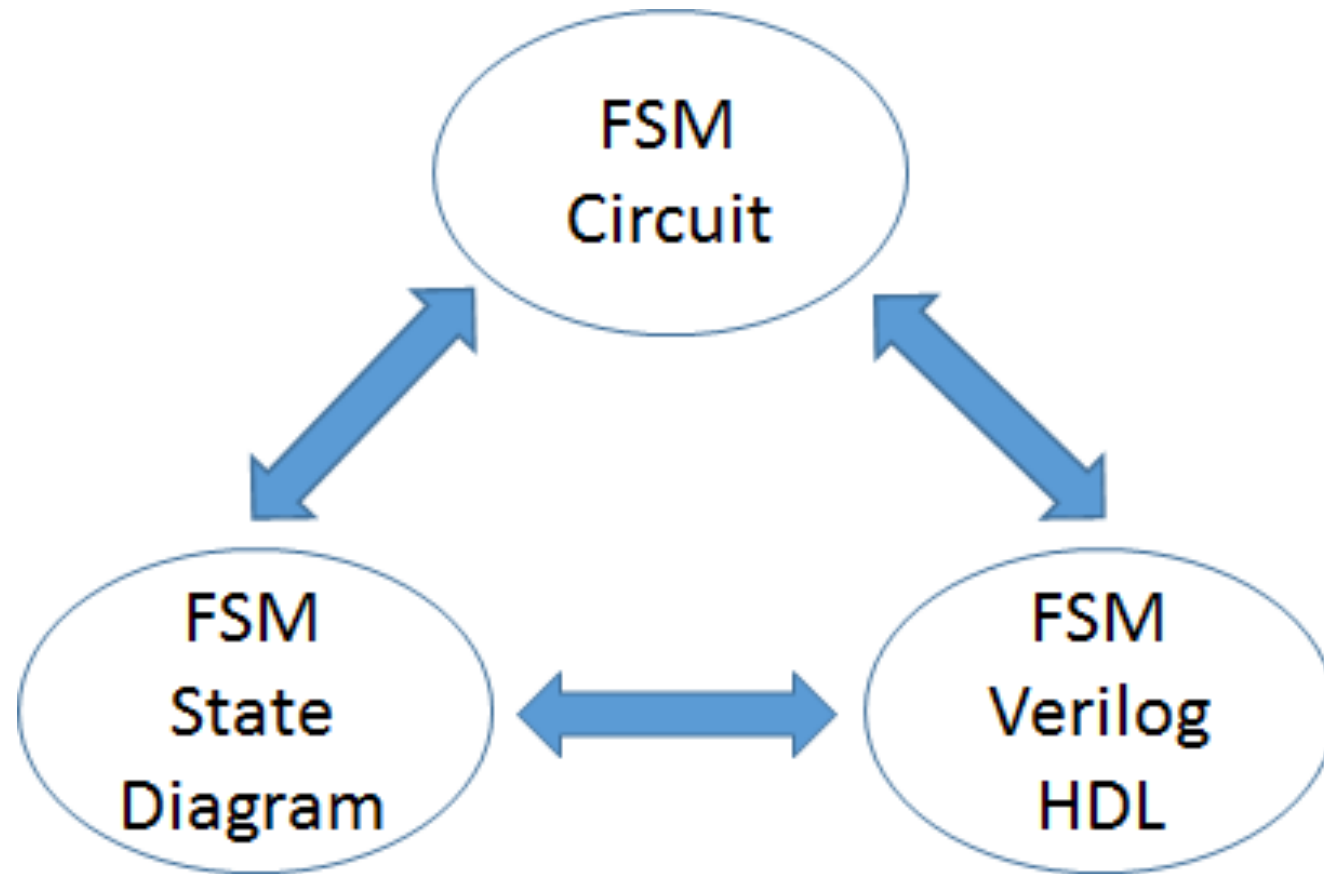
Moore: delayed assertion of P



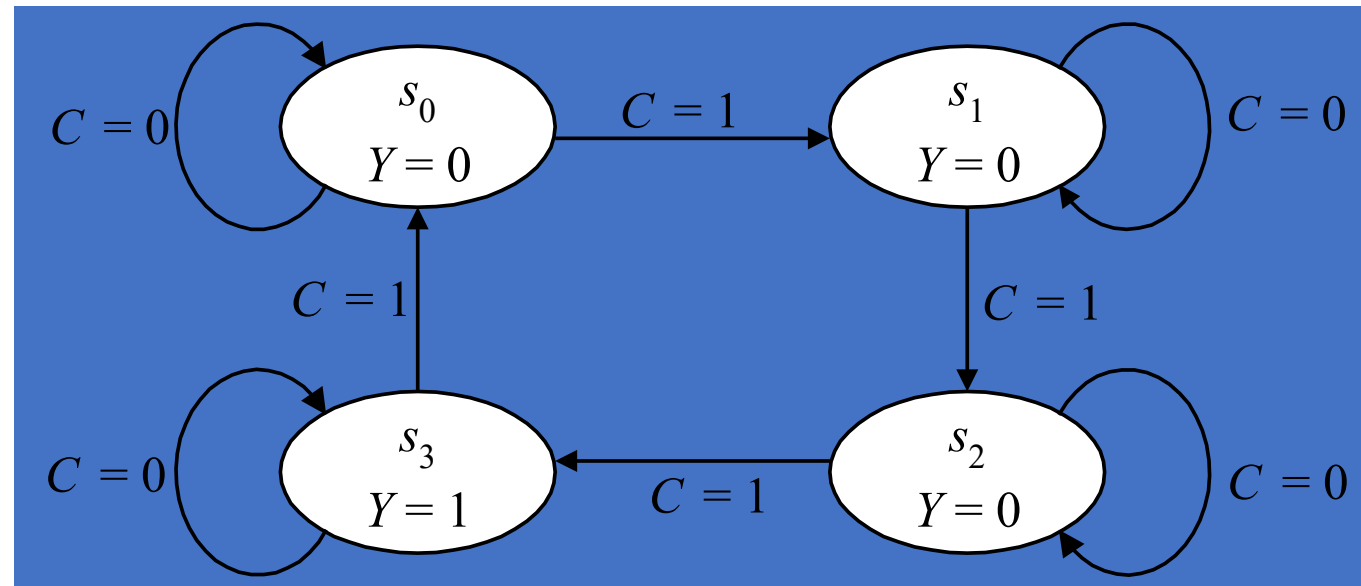
Mealy: immediate assertion of P



- Compared to a Moore FSM, a Mealy FSM might...
 - Be more difficult to conceptualize and design
 - Have fewer states



- Three paragraph Verilog codes



Verilog Code for FSM in three ways of coding



```
1 module one_step (clk, rst_n, C, Y);
2 input clk, rst_n, C;
3 output Y;
4
5 parameter [1:0] s0=2'b00, s1=2'b01;
6 parameter [1:0] s2=2'b11, s3=2'b10;
7
8 reg Y;
9 reg [1:0] sys_state;
10
11 always @(posedge clk or negedge rst_n)
12 if (!rst_n) begin
13     sys_state <= s0;
14     Y <= 0; end
15 else
16     case(sys_state)
17     s0: begin
18         if (C==0)
19             sys_state <= s0;
20         else
21             sys_state <= s1;
22         Y <= 0; end
23     s1: begin
24         if (C==0)
25             sys_state <= s1;
26         else
27             sys_state <= s2;
28         Y <= 0; end
29     s2: begin
30         if (C==0)
31             sys_state <= s2;
32         else
33             sys_state <= s3;
34         Y <= 0; end
35     s3: begin
36         if (C==0)
37             sys_state <= s3;
38         else
39             sys_state <= s0;
40         Y <= 1; end
41     default: begin
42         Y <= 0;
43         sys_state <= s0;
44     end
45 endcase
46 endmodule
```

```
1 module two_steps (clk, rst_n, C, Y);
2 input clk, rst_n, C;
3 output Y;
4
5 parameter [1:0] s0=2'b00, s1=2'b01;
6 parameter [1:0] s2=2'b11, s3=2'b10;
7
8 reg Y;
9 reg [1:0] current_state, next_state;
10
11 always @(posedge clk or negedge rst_n)
12 if (!rst_n) current_state <= s0;
13 else current_state <= next_state;
14
15 always @(current_state or C)
16 case (current_state)
17 s0: begin
18     if (C==0)
19         next_state = s0;
20     else
21         next_state = s1;
22     Y = 0; end
23 s1: begin
24     if (C==0)
25         next_state = s1;
26     else
27         next_state = s2;
28     Y = 0; end
29 s2: begin
30     if (C==0)
31         next_state = s2;
32     else
33         next_state = s3;
34     Y = 0; end
35 s3: begin
36     if (C==0)
37         next_state = s3;
38     else
39         next_state = s0;
40     Y = 1; end
41 default: begin
42     next_state = s0;
43     Y = 0; end
44 endcase
45 endmodule
```

```
1 module three_steps (clk, rst_n, C, Y);
2 input clk, rst_n, C;
3 output Y;
4
5 parameter [1:0] s0=2'b00, s1=2'b01;
6 parameter [1:0] s2=2'b11, s3=2'b10;
7
8 reg Y;
9 reg [1:0] current_state, next_state;
10
11 always @(posedge clk or negedge rst_n)
12 if (!rst_n) current_state <= s0;
13 else current_state <= next_state;
14
15 always @(posedge clk or negedge rst_n)
16 if (!rst_n) Y <= 1'b0;
17 else begin
18     case (next_state)
19     s0: Y <= 0;
20     s1: Y <= 0;
21     s2: Y <= 0;
22     s3: Y <= 1;
23     default: Y <= 0;
24 endcase
25 end
26
27 always @(current_state or C)
28 begin
29     case (current_state)
30     s0:
31         if (C==0) next_state = s0;
32         else next_state = s1;
33     s1:
34         if (C==0) next_state = s1;
35         else next_state = s2;
36     s2:
37         if (C==0) next_state = s2;
38         else next_state = s3;
39     s3:
40         if (C==0) next_state = s3;
41         else next_state = s0;
42     default:
43         next_state = s0;
44 endcase
45 end
46 endmodule
```

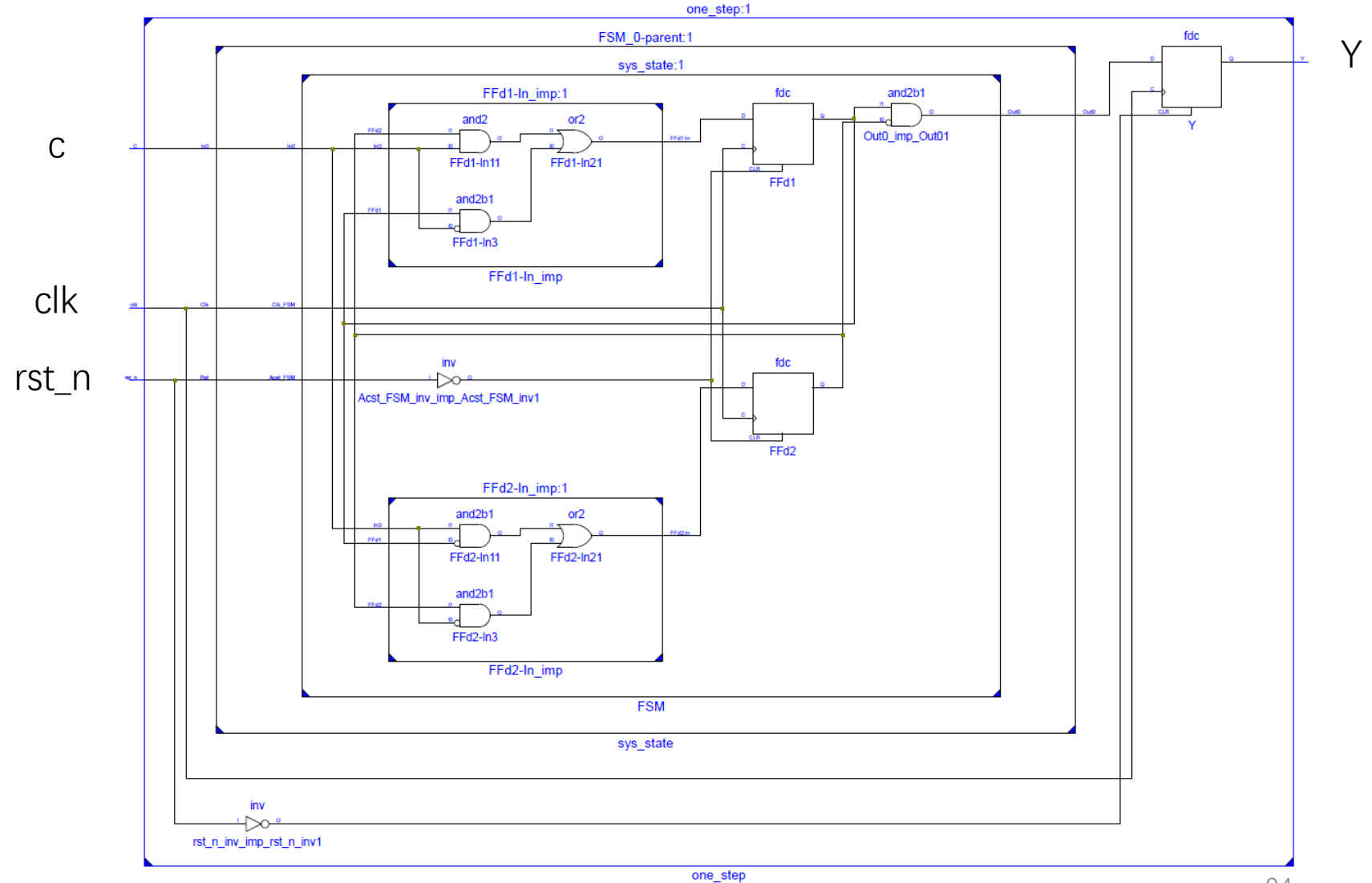
One-step FSM



```

1  module one_step (clk, rst_n, C, Y);
2  input clk, rst_n, C;
3  output Y;
4
5  parameter [1:0] s0=2'b00, s1=2'b01;
6  parameter [1:0] s2=2'b11, s3=2'b10;
7
8  reg Y;
9  reg [1:0] sys_state;
10
11 always @(posedge clk or negedge rst_n)
12   if (!rst_n) begin
13     sys_state <= s0;
14     Y <= 0; end
15   else
16     case(sys_state)
17     s0: begin
18       if (C==0)
19         sys_state <= s0;
20       else
21         sys_state <= s1;
22       Y <= 0; end
23     s1: begin
24       if (C==0)
25         sys_state <= s1;
26       else
27         sys_state <= s2;
28       Y <= 0; end
29     s2: begin
30       if (C==0)
31         sys_state <= s2;
32       else
33         sys_state <= s3;
34       Y <= 0; end
35     s3: begin
36       if (C==0)
37         sys_state <= s3;
38       else
39         sys_state <= s0;
40       Y <= 1; end
41     default: begin
42       Y <= 0;
43       sys_state <= s0;
44     end
45   endcase
46 endmodule

```





Is this circuit correct?

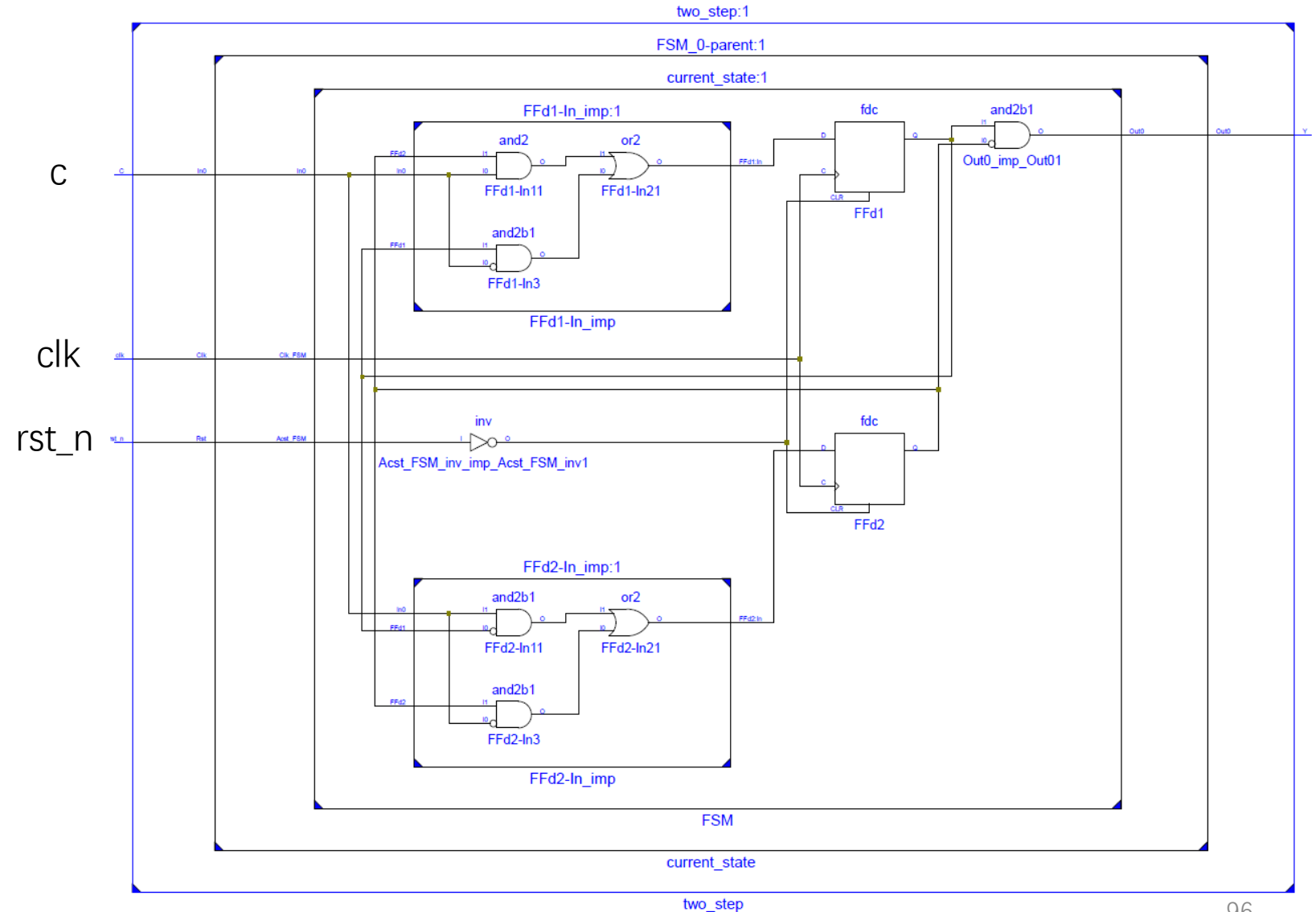
Two-steps FSM



```

1  module two_steps (clk, rst_n, C, Y);
2  input clk, rst_n, C;
3  output Y;
4
5  parameter [1:0] s0=2'b00, s1=2'b01;
6  parameter [1:0] s2=2'b11, s3=2'b10;
7
8  reg Y;
9  reg [1:0] current_state, next_state;
10
11 always @(posedge clk or negedge rst_n)
12     if(!rst_n) current_state <= s0;
13     else current_state <= next_state;
14
15 always @(current_state or C)
16     case (current_state)
17     s0: begin
18         if (C==0)
19             next_state = s0;
20         else
21             next_state = s1;
22         Y = 0; end
23     s1: begin
24         if (C==0)
25             next_state = s1;
26         else
27             next_state = s2;
28         Y = 0; end
29     s2: begin
30         if (C==0)
31             next_state = s2;
32         else
33             next_state = s3;
34         Y = 0; end
35     s3: begin
36         if (C==0)
37             next_state = s3;
38         else
39             next_state = s0;
40         Y = 1; end
41     default: begin
42         next_state = s0;
43         Y = 0; end
44     endcase
45 endmodule
46

```



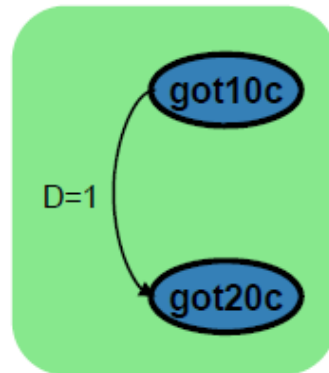
Y



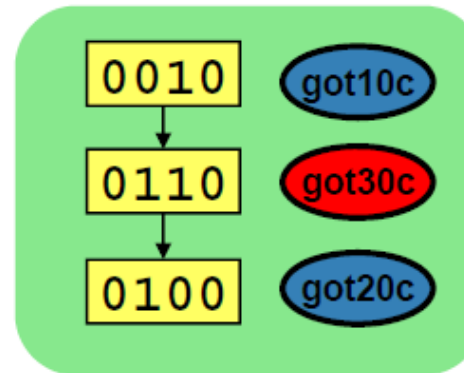
Is this circuit good enough?

- FSM state bits may not transition at precisely the same time
- Combinational logic for outputs may contain hazards
- Result: your FSM outputs may glitch!

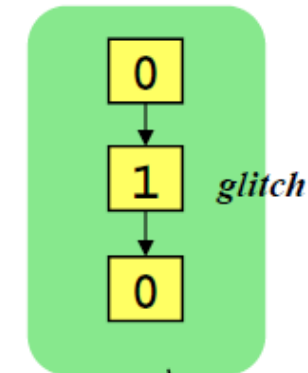
during this state transition...



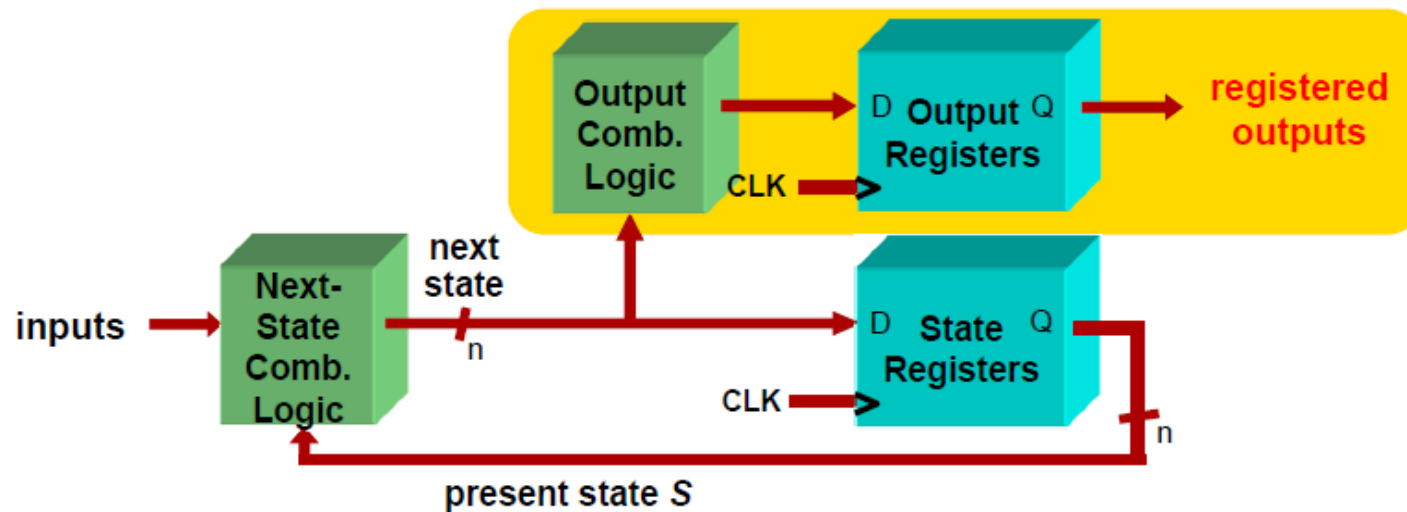
...the state registers may transition like this...



...causing the DC output to **glitch** like this!



Removing the Glitching



- Move output generation into the sequential always block
- Calculate outputs based on next state
- Delays outputs by one clock cycle. Problematic in some application.

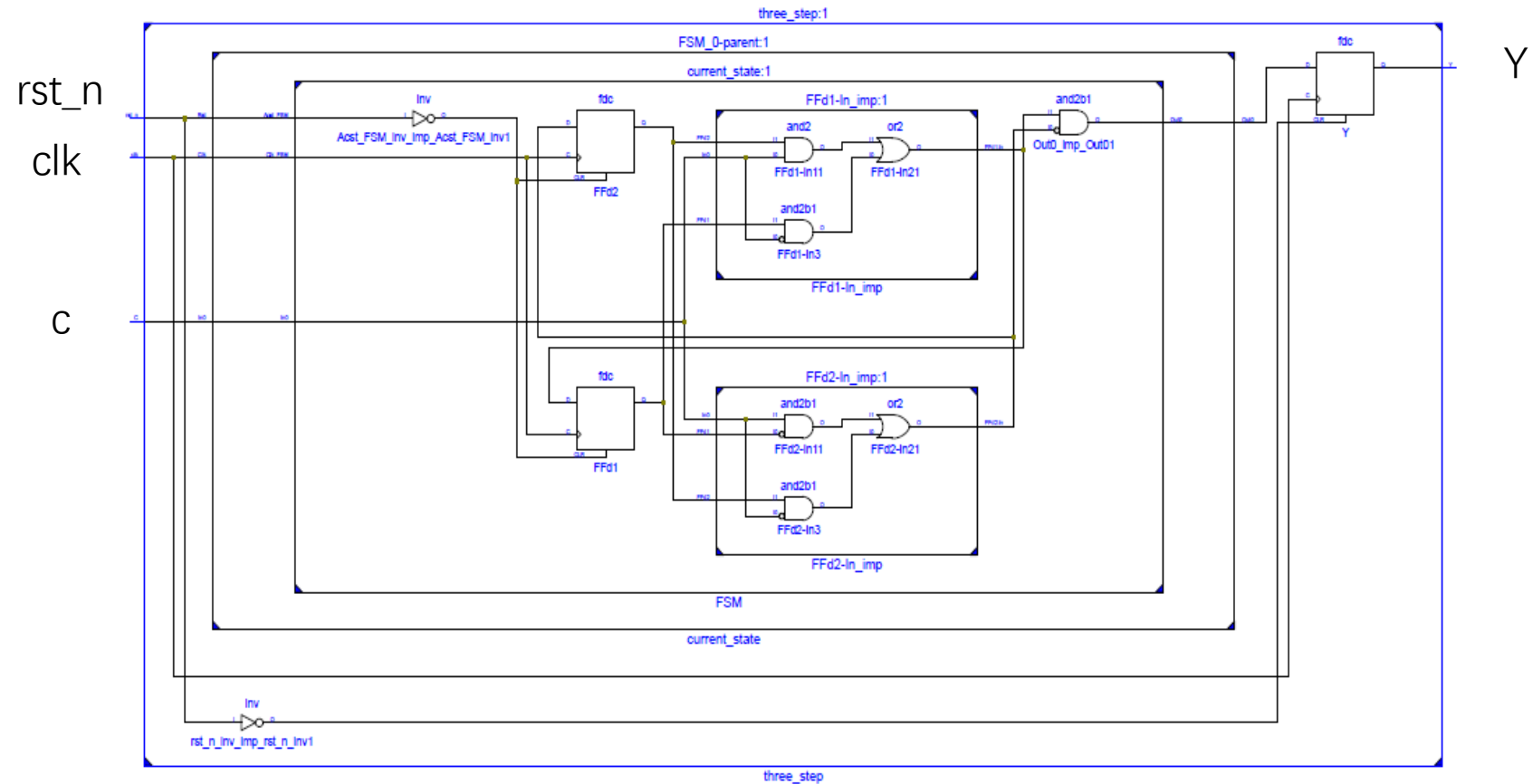
Three-steps FSM



```

1  module three_steps(clk, rst_n, C, Y);
2  input clk, rst_n, C;
3  output Y;
4
5  parameter [1:0] s0=2'b00, s1=2'b01;
6  parameter [1:0] s2=2'b11, s3=2'b10;
7
8  reg Y;
9  reg [1:0] current_state, next_state;
10
11 always @(posedge clk or negedge rst_n)
12     if(!rst_n) current_state <= s0;
13     else current_state <= next_state;
14
15 always @(posedge clk or negedge rst_n)
16     if(!rst_n) Y <= 1'b0;
17     else begin
18         case (next_state)
19             s0: Y <= 0;
20             s1: Y <= 0;
21             s2: Y <= 0;
22             s3: Y <= 1;
23             default: Y <= 0;
24         endcase
25     end
26
27 always @(current_state or C)
28     begin
29         case (current_state)
30             s0:
31                 if (C==0) next_state = s0;
32                 else next_state = s1;
33             s1:
34                 if (C==0) next_state = s1;
35                 else next_state = s2;
36             s2:
37                 if (C==0) next_state = s2;
38                 else next_state = s3;
39             s3:
40                 if (C==0) next_state = s3;
41                 else next_state = s0;
42             default:
43                 next_state = s0;
44         endcase
45     end
46 endmodule

```



If you manage to install Vivado Standard version on your computer,

- please try running the following RTL code and its testbench,
 - <https://vlsiverify.com/verilog/verilog-codes/moore-sequence-detector/>
 - <https://vlsiverify.com/verilog/verilog-codes/mealy-sequence-detector/>
- and modify the code such that
 - it adds one more state and one more output to the state machine (and does whatever you want),
 - both the code and testbench compile successfully and show me the waveform of your new design

If you haven't managed to install Vivado

- please do the same on <https://www.edaplayground.com/>, and take a snapshot of what your code and testbench being compiled successfully.



1. Derive the state diagram for an FSM that has an input w and an output z . The machine has to generate $z = 1$ when the previous four values of w were 1001 or 1111; otherwise, $z = 0$. Overlapping input patterns are allowed. An example of the desired behaviour is

w : 010111100110011111
 z : 000000100100010011

2. Write Verilog code for the FSM described here.